

SPEC CPU: THE NEXT GENERATION

Mahesh Madhav¹, Allen Lee², Andres Mejia³, Branden Moore⁴, Charan Soppadandi⁵, Chris Cambly⁶, Christoph Müllner⁷, Daniel Bowers⁸, David Reiner⁴, Denis Bakhvalov⁹, Di Zhao¹, Duane Voth⁴, Feng Xue¹, Frédérique Silber-Chaussumier¹⁰, James Bucek⁸, James Southern¹¹, Jiangning Liu¹, Jim Himer⁸, John Henning⁸, Kevin Smith¹, Kristen Yang⁴, Kunal Kashyap⁴, Mason Guy³, Mat Colgrove¹², Michael Berg¹³, Prasad Battini³, Prasad Joshi³, Rohit Prasad³, Shay Bhattacharya⁴, Sriyash Caculo¹, Stefan Reimbold⁶, Sundar Iyengar³, Van Smith⁴, and Zarko Todorovski⁶

¹Ampere Computing, ²IEIT, ³Intel, ⁴AMD, ⁵Dell Technologies, ⁶IBM, ⁷VRULL, ⁸SPEC, ⁹Rivos, ¹⁰ARM, ¹¹HPE, ¹²NVIDIA, ¹³SiFive

Abstract—The march toward developing relevant and robust CPU benchmarks continues with the introduction of SPEC CPU®2026, the next generation suite for measuring processor performance. This paper details the methodology behind its creation, showcasing a process centered on community collaboration and principled development. The suite is built upon a foundation of modern, open-source applications, selected and hardened through a process that emphasizes workload diversity, portability, and software longevity. A key contribution is Rolling Round-Robin Rate, a novel and standardized approach to running heterogeneous, multiprogrammed workloads that addresses a long-standing gap in benchmarking practice. Additionally, the suite features an expanded set of multithreaded benchmarks and introduces workloads with distinct microarchitectural profiles, reflecting the demands of contemporary software. By detailing our principled approach to benchmark selection, adaptation, and validation, we demonstrate how the SPEC CPU®2026 suite sets the standard for performance evaluation in the next era of computer architecture research and development.

I. INTRODUCTION

David Patterson’s principle that “benchmarks shape a field” posits that well-designed metrics accelerate progress by enabling fair and objective comparisons [1]. For more than three decades, the SPEC CPU suites have served this role, establishing a trusted lineage from CPU89 to CPU2017 [2–6]. This paper introduces SPEC CPU®2026, the newest iteration in this lineage, born from a multi-year effort to reflect the evolution of general-purpose computing. In an era dominated by discourse on specialized AI accelerators, the performance of general-purpose CPUs remains fundamental. The enduring importance of CPU microarchitectural innovation, distinct from advances in process technology or software, has been quantitatively demonstrated through longitudinal studies that rely on SPEC CPU data to normalize performance across generations [7]. SPEC CPU®2026 reaffirms its relevance by providing a refreshed and essential tool to measure performance and energy on this bedrock of modern computation.

To fulfill this role effectively, SPEC CPU adheres to a specific set of principles that define its scope. The suite exclusively encompasses natively compiled C, C++, and Fortran code, measuring performance in both single-threaded and multi-threaded scenarios. It does not cover managed runtimes (e.g., Java, Python, Julia), as the complexities of Just-In-Time

compilation introduce portability concerns and significant run-to-run variance that conflicts with SPEC’s foundational requirement for reproducibility [8–10]. This commitment to determinism is inseparable from a commitment to correctness. While some modern benchmark suites have only recently added result validation [11], SPEC CPU has, since its inception, not only measured how fast a workload runs, but how fast it runs *correctly*. This rigorous result validation is a hallmark of an industrial- and research-grade benchmark suite and remains central to its design.

This adherence to principle does not imply stagnation. On the contrary, the development of each new suite is an introspective process informed by lessons from the past and the evolving needs of the industry. Past suites were critiqued for having a limited number of microarchitectural behaviors [12] or for including workloads vulnerable to non-portable optimizations [13], feedback that SPEC has taken seriously.

SPEC CPU2026 is the direct result of this methodical, forward-looking process, embracing its role as a flexible research harness. It addresses past shortcomings and reflects contemporary software trends through several key enhancements. The suite is significantly larger, a deliberate strategy to increase application diversity and offer a variety of behaviors. Responding to the growing complexity of modern software, it introduces a new emphasis on front-end bound integer benchmarks characterized by large code footprints. A landmark improvement is the inclusion of multiple multithreaded integer benchmarks, addressing a critical gap in CPU2017. Finally, the suite introduces Rolling Round-Robin Rate (RRR), a new exhibition methodology provided to evaluate system performance with heterogeneous multiprogrammed benchmarks.

A fundamental role of SPEC CPU is to serve as a standardized tool for exploration, a principle occasionally misunderstood in external critiques. Some analyses [14] posit that SPEC CPU is arbitrarily vague or arbitrarily specific, based on incorrect assumptions that it mandates certain compiler flags or thread counts or operating systems. Such interpretations confuse the rules for formally compliant, *published* results with the suite’s broader purpose. The extensive configuration space is not forbidden; its exploration is simply required to be documented for transparency and reproducibility.

The most accurate analogy for SPEC CPU is that of a tape measure: it is a tool for measurement, not a dictate on what must be measured. While a small portion of the users submit official scores, the vast majority use the suite as a flexible harness for internal research on compilers, hardware, and software. Recognizing this primary use case, CPU2026 expands the suite’s analytical capabilities. For example, the raw output files have been augmented to include detailed statistics, such as variance and standard deviations. This evolution enhances the rigor of the already flexible harness, empowering users with deeper and more reliable data for their own explorations.

This paper details the principles of benchmark selection, the engineering efforts required to ensure portability and software longevity, and the key changes that define the new suite. By providing this transparent account, it demonstrates how SPEC CPU continues its long-standing mission: to provide a fair, relevant, and trusted standard for performance evaluation in the next era of computer architecture.

II. KEY CHANGES IN SPEC CPU2026

CPU2026 introduces a series of significant enhancements and strategic changes compared to its predecessor, CPU2017. These modifications are designed to reflect the evolution of modern hardware, contemporary software stacks, and principled benchmarking methodologies. The key changes are summarized below, with further details later in this paper.

Enhanced Suite Diversity. The composition of the suite has evolved strategically. The new suite deliberately elicits a broader set of microarchitecture responses than before, covers a wider span of application domains, and makes more extensive use of multiple workloads per benchmark. All the benchmarks can be seen in Table I.

Increased Scale and Resource Requirements. The suite is substantially larger, featuring more component benchmarks. To reflect the growing memory capacity of modern systems, the memory footprint of the SPECspeed® large multi-threaded benchmark suite has increased from 16 GB to 64 GB. The SPECrate® throughput benchmark suite’s footprint remains the same at 2 GB per copy. Each benchmark retains `test` and `train` sizes, which have much shorter runtimes than the measured and reported `ref` size.

Updated Language Standards. The suite adopts modern ISO standards: C18 (from C99), C++17 (from C++03), and Fortran 2018 (from Fortran 2003). In addition to OpenMP, language parallelism is employed through C++’s `std::thread` and Fortran’s `DO_CONCURRENT`. Some floating point benchmarks require precise math for functionality and verification, and thus may not work correctly with fast/relaxed math.

New Analytical Capabilities. The harness and reporting tools have been enhanced. The raw output now includes additional population statistics for SPECrate such as coefficient of variation, quartiles, min/max/average copy times, and standard deviation, enabling more rigorous analysis of multi-copy runs. Additionally, a new exhibition run style, Rolling Round-Robin

Rate (RRR), has been introduced to facilitate systems research with heterogeneous, multiprogrammed workloads.

Expanded Reporting Categories. To better represent real-world deployment scenarios, two major reporting categories have been added. First, official, compliant scores can now be submitted from bare-metal instances on public cloud platforms, moving beyond the “estimated” status of the past. Second, a new category distinguishes between results obtained using vendor-supported and community-supported open-source compilers. This is motivated by the widespread use of community compilers like GCC and LLVM, and their distinct performance characteristics compared to vendor compilers [53–56], providing a more representative view of the performance users experience on different software stacks.

III. BENCHMARK DEVELOPMENT

The benchmark development pipeline commenced by soliciting candidates from the open-source community, academic researchers, and industry practitioners. This stage was conducted through the CPUv8 search program [57], which ran from February 2020 to March 2023. Candidates then progressed through the steps of adaptation into a formal benchmark, workload selection, and performance characterization, before being considered for final selection. It is important to note that this pipeline is not strictly linear, as these stages ran concurrently: new candidates are continually introduced while others mature or are culled from the process.

A. Search Program

The CPUv8 benchmark search program proved remarkably successful, drawing 33 benchmark candidates into consideration. An impressive 29 of these completed initial porting and workload definition [58] to advance into the benchmark selection stage, with 24 of those external candidates ultimately integrated into the final suite (Table I). A significant portion of these submissions originated from projects with strong open-source community backing. This offered collaborations with authors and their communities, which became essential for cross-system porting and for addressing issues discovered during the development and testing process. The resulting benchmarks are diverse and meaningful, including prize-winning drug discovery programs vital to COVID-19 vaccine research [59], flight simulators used by government agencies [60], brain modeling tools [61], and even a media application that won an Academy Award [62]. The intensive evaluation process didn’t just fortify these applications into SPEC CPU benchmarks; it also led to valuable fixes and enhancements that were upstreamed back to the original projects. The development process stands as a powerful testament to the symbiotic relationship between SPEC and the open-source world.

B. Adaptation

The development of the SPEC CPU suite has always been guided by a core philosophy that prioritizes the selection of real-world applications with significant user bases. While purpose-built microbenchmarks can be valuable for exercising

TABLE I: The SPEC CPU®2026 Benchmarks

SPECrate®2026 Integer	SPECspeed®2026 Integer	speed MT ^[A]	Language	KLOC files ^[B]	KLOC hit ^[C]	Application Domain	ref
	801.xz_s	MT	C++, C	53	5	Data compression	[15]
706.stockfish_r			C++	13	5	Game (chess) - A/B search, deep learning neural network	[16]
707.ntest_r	807.ntest_s	MT	C++	16	5	Game (othello) - A/B search with heuristic eval function	[17]
708.sqlite_r			C	245	23	SQL compiler/interpreter and database	[18]
710.omnetpp_r			C++, C	224	19	Discrete event modeling - network and queuing simulations	[19]
714.cpython_r			C	747	56	Python interpreter	[20]
	817.flac_s	MT	C++, C	57	7	Lossless audio codec	[21]
721.gcc_r	821.gcc_s	MT	C++, C	3,833	326	C language optimizing compiler	[22]
723.llvm_r	823.llvm_s	MT	C++, C	3,167	123	C/C++ language optimizing compiler	[23]
727.cppcheck_r	827.cppcheck_s	MT	C++	287	111	Static analysis of C/C++ code	[24]
729.abc_r	829.abc_s		C++, C	989	37	Sequential logic synthesis and formal verification	[25]
734.vpr_r	834.vpr_s		C++, C	210	30	FPGA circuit place and route	[26]
735.gem5_r	835.gem5_s		C++, C	971	78	Computer architecture simulation model	[27]
	838.diamond_s	MT	C++, C	239	12	Bioinformatics - metagenomics and protein sequencing	[28]
	846.minizinc_s	MT	C++, C	372	33	Constraint programming (solvers: gecode and chuffed)	[29]
750.sealcrypto_r			C++, C	39	5	Security and privacy - Homomorphically Encrypted query	[30]
753.ns3_r	853.ns3_s		C++	942	71	Discrete event network simulator for internet systems	[31]
	854.graph500_s	MT	C	10	1	Graph analytics	[32]
777.zstd_r			C	58	7	Data compression/decompression	[33]

SPECrate®2026 Floating Point	SPECspeed®2026 Floating Point	speed MT ^[A]	Language	KLOC files ^[B]	KLOC hit ^[C]	Application Domain	ref
	800.pot3d_s	MT	Fortran	12	1	Solar physics: finite diff method, conjugate gradient solver	[34]
	803.sph_exa_s	MT	C++	3	1	Astrophysics - Smoothed Particle Hydrodynamics (SPH)	[35]
709.cactus_r	809.cactus_s	MT	C++, C	187	19	Astrophysics - relativity, finite difference, time integration	[36]
	811.tealeaf_s	MT	C	5	1	High energy physics	[37]
	816.nab_s	MT	C	26	2	Molecular modeling	[38]
	820.cloverleaf_s	MT	Fortran	10	1	Explicit hydrodynamics	[39]
722.palm_r	822.palm_s	MT	Fortran	218	6	Atmospheric science	[40]
731.astcenc_r			C++	43	8	Computer vision - Adaptive Scalable Texture Compression	[41]
736.ocio_r			C++	183	13	Color management for visual effects and animation	[42]
737.gmsh_r			C++, C	721	35	Finite element mesh generation	[43]
748.flightdm_r			C++	100	14	Flight dynamics models for aeronautics	[44]
749.fotonik3d_r	849.fotonik3d_s	MT	Fortran	15	2	Computational Electromagnetics (CEM)	[45]
	857.namd_s	MT	C++	9	2	Classical molecular dynamics simulation	[46]
765.roms_r	865.roms_s	MT	Fortran	585	13	Regional ocean modeling	[47]
766.femflow_r			C++	2,505	23	Fluid dynamics: high-order finite element method	[48]
767.nest_r	867.nest_s	MT	C++	208	17	Neuroscience simulator for spiking neural network models	[49]
772.marian_r	872.marian_s	MT	C++	219	15	Neural machine translation for written language	[50]
782.lbm_r			C	1	1	Computational fluid dynamics, Lattice Boltzmann Method	[51]
	881.neutron_s	MT	C	4	1	Physics simulation of neutron transport in nuclear reactors	[52]

^[A] MT indicates the SPECspeed benchmark uses parallelism; either multi-threading or multi-tasking.

^[B] KLOC = line count in thousands. Counts all files in the source directory; includes comments and blank lines.

^[C] KLOC = line count in thousands. Only counts lines which are exercised based on benchmark code coverage metrics.

specific CPU features, the committee intentionally biases towards production software used in the field. This approach ensures that the performance characteristics measured are representative of genuine computational workloads, and that optimization of the hardware systems running these benchmarks does indeed help improve performance in the community. Transitioning a real-world application into a trusted benchmark requires an adaptation process to satisfy the non-negotiable principles of determinism, reproducibility, and portability.

This adaptation can be analogized to studying an organism in a controlled environment versus its native habitat. To enable systematic and reproducible study, the *organism* of the application must be carefully adapted to the *laboratory* of the SPEC CPU harness. This process involves a series of modifications designed to eliminate external sources of variance and ensure that the benchmark clearly measures the performance of the System Under Test (SUT), and not the surrounding environment. The fundamental goal is to ensure that the benchmark executes an identical amount of user-space work across any compliant system, and produces a identical result on every run within a given tolerance. To achieve this level of rigor, each candidate benchmark undergoes a series of modifications:

Elimination of Non-Determinism. Sources of high entropy like reads from `/dev/random`, or calls to hardware-based true random number generators, are replaced with deterministic pseudo-random number generators like the Mersenne Twister `std::mt19937`. Additionally, the C++17 standard does not define strict results for `std::sort`, so we convert those usages to `std::stable_sort` which is guaranteed to produce the same results across library implementations; and likewise for other unstable standard algorithms that have stable equivalents. These compromises provide reproducibility in both application control logic and data results, from run to run and across hardware systems and compiler libraries.

Maximization of Portability. All platform-specific code, including hand-coded assembly and compiler intrinsics, is removed and replaced with portable C, C++, or Fortran equivalents (§V). This is often the most significant deviation from the original application but is essential for ensuring forward and backward compatibility of the suite across decades of architectures. Consequently, benchmark candidates with a heavy reliance on non-portable, hand-optimized code are generally disfavored and do not make it far in the selection process.

Isolation from the Environment. The benchmark is decoupled from its external execution environment. This includes removing calls that query or modify the environment (`getenv`, `setenv`), eliminating internal measurement and control (`gettimeofday`, `getrusage`, `setrlimit`), and excising any debugging hooks that could alter program behavior (`signal`, `sigaction`, `dlopen`).

Focus on User-Space Execution. To ensure the benchmark primarily measures CPU and memory subsystem performance rather than OS efficiency, system calls are minimized. The target is for at least 95% of the execution time to be spent

within the user-space code provided by the benchmark. Exceptions are consciously made for ubiquitous library functions (`malloc`, `strcpy`, standard math functions), as their performance is of broad interest to the community.

Suppression of Threading Artifacts. For single-threaded SPECrate benchmarks derived from multithreaded applications, synchronization primitives like locks and mutexes introduce a “threading tax” that is irrelevant to measuring single-threaded performance. This overhead is systematically identified and suppressed for SPECrate builds, while the necessary hooks are preserved for use in the multi-threaded SPECspeed version of the benchmark.

Validation Across Multiple Systems. The committee runs a continuous integration process through kit build and testing which involves extensive validation across a matrix of hardware, operating systems, and compilers. The committee membership consists of representatives from companies that implement the x86, ARM, POWER, and RISC-V architectures; their products are used to exercise the candidate benchmarks under Linux, Windows, macOS, and other operating systems. We build the codes using the latest open-source compilers, GCC and LLVM, as well as vendor compilers from Intel, AMD, IBM, NVidia, HP/Cray, and Microsoft. Each of these systems must produce the same answer, within tolerance, of the golden reference results to be considered a successful run. This exhaustive testing at multiple optimization levels (`-O2`, `-O3`, LTO, PGO) validates the benchmark’s portability fitness.

Legal. Finally, a thorough legal review is conducted on all included source code and data inputs. Every file must have its provenance cited and be licensed correctly, to ensure the suite is commercially distributable on solid legal foundations [63].

C. Floating-Point versus Integer

Traditionally, SPEC CPU is split into two suites: integer (INT) and floating-point (FP). Although some may consider this as a throwback from an era when floating-point units (FPUs) were optional co-processors, the bifurcation actually began with SPEC CPU2000. So, although CPUs from that era already integrated powerful FPUs, the suites were created to distinguish workload domains. In the modern era, this can be seen where INT versus FP tends to correlate loosely with cloud computing versus high-performance computing (HPC). Since 2000, SPEC has employed a quantitative criterion for this classification: applications with over 10% of their dynamic instructions being floating-point were designated FP, while those with less than 1% were designated INT.

This clear delineation, effective through the CPU 2017 suite, has become increasingly blurred by architectural evolution. In contemporary processors, integer and floating-point SIMD execution units are often co-located or share scheduling resources. Moreover, it is now common for predominantly integer-based SIMD applications to leverage floating-point load/store instructions for vector memory accesses, which complicates instruction-based accounting; the counts can also differ significantly based on compiler optimization levels [3].

This architectural and programmatic convergence has created a significant “gray zone” for applications that exhibit an FP instruction composition between 1% and 10%.

A substantial number of open-source candidates for the CPU2026 suite fell squarely into this ambiguous category, possessing characteristics of integer workloads while incorporating a non-trivial amount of floating-point operations. To resolve this, the SPEC committee adopted a qualitative methodology, applied on a case-by-case basis. For each application within the gray zone, the final classification was determined by a pragmatic assessment based on the application’s primary computational purpose and its established reputation within its user community as either an integer or floating-point workload.

D. Workload Selection

The performance profile of an application is highly sensitive to its input workloads, which encompass command-line arguments, configuration files, and datasets. The SPEC CPU suite has consistently biased its selection towards real-world datasets that are relevant and representative of common usage in the field. The challenge of selecting a single, representative input from a near-infinite space is well-documented [64, 65]. To address this, SPEC heavily relies on the guidance of benchmark authors and their respective user communities, a principle institutionalized through the search program (§III-A). This collaborative approach ensures that the selected workloads are authentic and accurately reflect the benchmark’s intended application domain.

One objective for CPU2026 was to enhance the suite’s resilience to targeted, non-generalizable optimizations. This goal was pursued by expanding the use of multiple, distinct workloads for a single benchmark—a capability present in prior suites but more systematically employed in this version. This multi-workload approach serves several purposes: it increases the diversity of exercised code paths, captures a wider range of application behaviors, and makes the benchmark more difficult to crack, in that a compiler or hardware optimization might yield a significant speedup on one specific input but fail to generalize across the others. The diversity among these sub-workloads is qualitatively visualized in the Basic Block Vector (BBV) plots explained in Figure 1 and presented in the Appendix¹ for all single-threaded benchmarks, where one can compare the execution profiles across inputs.

This methodological enhancement, however, introduces a notable trade-off for performance analysis. By design, the aggregate score for a multi-workload benchmark represents a composite of behaviors, which could obscure and dilute the characteristics of its individual components. For instance, 729.abc and 727.cppcheck contain a mix of core-bound, high-IPC workloads and memory-bound, low-IPC workloads. Consequently, the aggregate benchmarks cannot be singularly categorized as high-bandwidth or high-IPC, as their subcomponents exhibit a multitude of behaviors. To support deeper,

more granular analysis and to empower the research community, each benchmark’s documentation includes instructions for creating custom input sets. This enables further investigation, in the spirit of research projects such as the Alberta Workloads [67].

IV. PRINCIPLES OF BENCHMARK SELECTION

The methodology behind the SPEC CPU suite has been recognized as a best practice in performance evaluation, with adjacent benchmarking communities either adopting similar principles or validating the approach [68, 69]. This section details the foundational tenets of crafting the suite.

A. Culling

The selection of benchmarks for a SPEC CPU suite is a rigorous, multi-faceted process designed to curate a balanced, relevant, and scientifically sound collection of workloads. While many promising applications are proposed as candidates, a process of principled attrition is necessary to ensure the final suite meets stringent quality standards. The primary criteria that led to excluding candidates are given here (with concrete examples offered in §IV-D). The first two rationales listed below are fundamental, and the remaining are based on suite-level composition, logistical, and technical considerations.

Determinism. A foundational requirement for any SPEC benchmark is the execution of a deterministic and reproducible quantum of work. This principle was a critical filter for applications based on heuristic search algorithms, such as linear solvers, constraint programming, or those employing gradient descent. These programs often exhibit non-deterministic execution paths, where minor architectural or compiler differences can lead to “short-cuts” to the solution. As this violates the core tenet of equal work across all test platforms, such candidates, while valuable in their own right, are unsuitable for comparative CPU benchmarking. If deterministic execution was not possible, the candidate was excluded.

Development Divergence. A second essential factor is the benchmark’s representativeness of its real-world counterpart after undergoing necessary modifications for portability. To ensure broad compatibility, all platform-specific code, such as intrinsics and hand-tuned assembly, must be removed. For certain domains, particularly modern AI and media encoding applications, this “defanging” process caused the benchmark’s performance profile to diverge significantly from the original highly optimized application. When the resulting portable code no longer reflected the computational characteristics of the software used in the field, its value as a representative benchmark was diminished, leading to its removal.

Domain Redundancy and Scope. To ensure broad coverage, the suite cannot be over-represented by a single application domain. In fields with multiple high-quality candidates (e.g., file compression, scripting languages), a “horse race” ensued, with only the most prominent or relevant candidate being selected. Conversely, applications with an exceptionally narrow user base that did not address broad industry challenges were deemed too specialized for inclusion.

¹Please see the extended edition of this paper, available on arXiv [66].

Codebase Health and Maintainability. Preference was given to modern, actively maintained codebases. Candidates based on decades-old, unmaintained code were rejected as they are poor indicators of future computational trends. All code must be clear, well-structured, and maintainable by the committee for the suite’s lifespan. Obfuscated or unwieldy code presents an unacceptable maintenance burden.

Insufficient Maturity. Some candidates, though promising, were not sufficiently developed to meet the production timeline and were deferred for future consideration. Usually this was due to lack of portability across systems, which required more development and testing effort.

Excessive I/O or “Peak” Profiles. The suite is designed to measure CPU and memory performance. Candidates with significant file I/O were rejected as their performance would be unduly influenced by storage subsystems and OS calls [70]. Similarly, benchmarks with “peak” profiles [71], where runtime is dominated by a few functions, were disfavored due to their vulnerability to narrow or non-portable compiler optimizations that would not benefit general-purpose computing.

Potential Bias. To maintain objectivity, candidates perceived as being pre-tuned for a specific committee member’s architecture are heavily scrutinized and often excluded to prevent inherent bias. The development process itself is conducted transparently, ensuring there is no intentional attempt to mislead or obfuscate the benchmark’s behavior.

B. Selection

Despite SPEC’s historical foundation of selecting benchmarks from real-world, portable source code, a persistent misconception is that the suite is “synthetic” [72], or not composed of “real workloads” [73]. This paper directly addresses that perception. The final selection process is guided by principles designed to curate a balanced, diverse, and forward-looking suite, rooted in a transparent and open development culture where proprietary interests are subordinated to the creation of technically credible and vendor-neutral benchmarks [74].

The selection process was also driven by a goal to enhance the suite’s behavioral diversity and address known gaps from previous versions [75]. A notable outcome in the integer suite is the inclusion of benchmarks bottlenecked by the CPU’s front-end, characterized by heavy instruction delivery pressure, high ITLB miss rates, or frequent branch mispredictions. This shift reflects the changing landscape of modern software, which increasingly features large code footprints and complex control flow, moving beyond just the predominantly back-end-bound workloads of previous suites [76].

C. Number of Benchmarks

A defining characteristic of CPU2026 is an expanded benchmark count of 52 (up from 43 in CPU2017), made possible by the prolific outcome of the search program (§III-A). The ability to craft a larger suite provided a chance to address key challenges observed in the academic and industrial use of previous suites. Historically, concerns have been raised

about researchers creating subsets of benchmarks from inside and outside of the suite without a clear justification [77], a practice which confounds reviewers and severely hinders the direct comparison of results across studies [78]. Furthermore, past suites have been critiqued for potential redundancy in workload behavior [79–81]. By increasing the number of benchmarks, we aimed for a greater diversity of microarchitectural behaviors, programming styles, and application domains. A larger, more diverse suite offers more optimization problems to solve, diminishes the impact of “cracking” a single benchmark via the 1/N rule, and disincentivizes non-portable, benchmark-specific optimizations.

The expansion introduces trade-offs, namely analysis complexity. While individual benchmarks are shorter to keep the total runtime of the suite comparable to that of CPU2017, the increased analysis volume is an intentional feature. It steers hardware and compiler systems designers away from narrow tuning and toward developing more general-purpose optimizations that ultimately deliver greater uplift to end-users.

D. Culled Benchmark Candidates

In response to reviewer feedback requesting concrete examples, this section details the rationale behind the exclusion of several notable candidates and application domains. These stories illustrate the practical application of the selection principles outlined in Section §IV-A.

Modern AI Workloads. We evaluated portable CPU inference engines from the transformer/LLM era, including llama.cpp [82] and whisper.cpp [83]. These candidates advanced deep into the evaluation process due to the domain’s importance. However, restricting them to portable C++ codepaths (with intrinsics removed) caused a fundamental divergence from their real-world behavior. The resulting benchmarks became uncharacteristically compute-bound, with longer instruction path lengths between transactions and significantly lower MPKI than their field-deployed counterparts. Both devolved into workloads where 95% of the runtime was spent in a single, inefficient hot loop. Optimizations on such a narrow, unrepresentative loop would offer no value to the industry. Furthermore, llama.cpp faced significant challenges with result verification across systems, requiring us to “put inference on rails”—forcing determinism by capturing the sequence of generated tokens offline and then forward-feeding them back into the model during runtime—a process which further separated the candidate from realistic use.

Cryptography. While a critical CPU workload, production cryptography (e.g., AES, RSA) is dominated by hand-tuned assembly and ISA intrinsics. Removing these architecture-specific hooks to create a portable benchmark results in code that is not representative of real-world deployments. A generic, unoptimized crypto workload would be a poor proxy for what is run in the field. We did retain 750.sealcrypto (homomorphic encryption) because its core exercises finite-field mathematics, which is an algorithmic phase that will remain relevant to general-purpose CPUs even as other parts of the HE stack migrate to specialized hardware.

Media Codecs. We considered AV1/AOM [84] and Opus [85], two codecs used widely for internet video and audio. Both make such heavy use of architecture-specific assembly that removing these implementations rendered their performance profiles unrepresentative. And in the case of Opus, the program processed audio so quickly that the workload shifted from CPU-bound to I/O-bound, with multi-copy SPECrate runs stalling on disk activity while the CPU remained idle. 817.flac doesn't have this issue since the benchmark is a multi-threaded program writing into just a single file.

Key-Value Stores. A high performance key-value store library was adapted as a benchmark and reached the final stages of selection. After extensive experimentation, the committee concluded that the benchmark's workload did not reflect real-world deployments of that application, as it exhibited pathological behavior such as abnormal memory bandwidth usage and redundant data decompression. Various different configurations were attempted to alleviate these issues, but it became clear that none of the options could represent field usage meaningfully. As we ran out of time for more development, this database candidate was dropped.

Duplicate Application Domains and Behaviors. We evaluated four high-quality data compression candidates (xz, brotli [86], 7-zip [87], zstd). While compression is an important domain, the performance behaviors of these four were highly similar, creating redundancy. We selected 777.zstd for the intrate suite due to its widespread adoption (Linux kernel, cloud services, databases). We retained 801.xz for the intspeed suite because it was the only compression candidate that offered a multithreaded implementation.

Non-Determinism and the “Equal Work” Principle. Some candidates used search or non-linear optimization, where different platforms can complete different amounts of work while still arriving at the same solution. For example, 737.gmsh originally included an adaptive mesh refinement phase whose iteration count could vary by 30% depending on FP numerics, compiler flags, or ISA. By disabling just the adaptive phase of meshing, we successfully created a fully deterministic workload while still retaining field behavior. In contrast, the candidate HiGHS [88] is a linear programming solver that could not be similarly adapted. After consultation with its community [89], it was determined that guaranteeing equal work would require using a trivial input problem with a single solution path, resulting in a workload which is not representative of field behavior. A different approach proved successful for the 846.minizinc constraint solver benchmark, which avoids this pitfall by using unsatisfiable problem inputs. When no solutions exist, the solver must exhaustively search all paths, naturally ensuring equal work on all systems.

Legacy Kernels and Bespoke Microbenchmarks. A proposal to include the NASA Parallel Benchmarks [90] was declined. While historically significant, these kernels are now over three decades old, and SPEC prioritizes modern, actively maintained codebases. Similarly, a microbenchmark inspired by the financial services industry was rejected. Although the

domain is of interest, the small code did not represent modern FSI algorithms, and SPEC disfavors member-authored bespoke benchmarks to avoid any perception of bias.

Virtualization and containers. These environments are important and growing, but SPEC CPU measures natively compiled, application-level CPU and memory behavior under a single-workload harness. Virtualization adds system-level effects and policies that fit better under other SPEC suites and methodologies, such as SPECvirt®2021 [91].

Looking Forward. These experiences highlight a central challenge: balancing portability with representativeness for workloads that rely on ISA-specific optimizations. For future suites, the committee will evaluate whether allowing optional, architecture-specific libraries alongside a generic reference implementation could enable the inclusion of these important modern applications without compromising the suite's core principles of equal work, portability, and vendor neutrality.

V. LONGEVITY AND PORTABILITY

After the benchmarks were selected, the focus turned to another mainstay philosophy of SPEC CPU, namely portability. This principle is upheld through two key practices: methodical adherence to ISO language standards and a commitment to high-quality, warning-free code. This approach is the primary reason for the suite's exceptional longevity, as evidenced by suites like SPEC CPU2000 remaining relevant a quarter-century after their release, particularly in embedded systems.

By ensuring both forward compatibility (allowing older suites on new systems) and backward compatibility (enabling new suites on legacy hardware), this standards-based design preserves SPEC CPU's value across decades. The CPU2026 suite continues this tradition with its baseline of C++17, C18, and Fortran 2018. The following subsections detail the specific development practices and validation efforts essential to upholding these principles.

A. Code Hardening and Standards Compliance

This commitment to standards is enforced through a multifaceted code hardening process. The goal is not merely to compile the code, but to ensure it is robust, warning-free under pedantic mode (`-Wpedantic`), and free of ambiguous behavior. This ensures that a standards-compliant compiler developed decades from now will be able to build the suite. This process addressed several common categories of issues:

Elimination of Undefined Behavior. A primary focus was the elimination of undefined, unspecified, and implementation-defined behavior, which can lead to non-deterministic results or outright failures across different compilers and platforms. This was achieved through both static analysis via compiler warnings and dynamic analysis using runtime sanitizers. These efforts rectified critical issues such as incompatible types which resulted in dangerous pointer conversions [92] and data overflows [93]; uninitialized member variables that led to divergent behavior and segmentation faults [94–96]; and subtle object construction races like the “initialization-order fiasco” detected by Address Sanitizer [97].

Ensuring Data Model Portability. A second critical task was to ensure the code was agnostic to platform-specific data models. This involved correcting a class of warnings related to mismatched integer types and signed/unsigned comparisons, often by standardizing on consistent types for object sizes and indices [98]. Mismatches in type usage were also corrected to prevent portability issues, for example on platforms where fundamental types have different sizes [99].

Modernization to C++17 Standards. The process also involved modernizing legacy codebases to comply with the selected C++17 standard, which required addressing a wide range of issues identified by modern, standards-compliant compilers [100, 101]. Specific modernization efforts included replacing deprecated features such as `std::bind2nd` with modern lambda functions, converting `std::random_shuffle` to the newer `std::shuffle` with a deterministic random engine, removing the now-obsolete `register` keyword [102], and standardizing the use of `std::nan` in place of custom NaN implementations [103].

Removal of Non-Standard Language Extensions. Finally, to guarantee maximal portability across all compliant compilers, non-standard language features and compiler-specific extensions were spliced out. This included replacing compiler-specific attributes like ‘always_inline’ with the standard ‘inline’ keyword, and removing uses of the non-standard ‘restrict’ keyword. This adherence to the ISO standard ensures that the performance of the benchmarks is not dependent on proprietary features that favor a particular compiler.

B. Endianness

To ensure the broad applicability and architectural neutrality of the CPU2026 suite, a dedicated validation effort was undertaken to guarantee portability to big-endian systems. The primary platform for this validation was IBM AIX running on the POWER architecture. This process uncovered and led to the resolution of several classes of portability issues, with many of the resulting patches being upstreamed to the originating open-source communities [104–107].

Endian-Dependent Input Data Formats. Several benchmarks assumed a little-endian format for their on-disk input files, requiring modifications to ensure data could be correctly interpreted on big-endian systems. For instance, the 731.astcenc benchmark initially failed because its input textures were encoded using a third-party library lacking big-endian support. The resolution involved transitioning the input data to a new, endian-agnostic format. Similarly, 772.marian presumed a little-endian layout for its model files, which was rectified by integrating byte-swapping routines into the data loading process for big-endian systems.

Memory Layout and Type-Punning Assumptions. A common class of errors stemmed from C/C++ code that made implicit assumptions about the in-memory byte order of data structures, often through pointer casting and dereferencing. A verification failure in 748.flightdm was traced to an unsafe

type-cast that violated memory layout assumptions on big-endian systems. Another in 846.minizinc manifested due to layout of bit fields in union types where the union held either pointers or numbers, and these fields fell out of alignment. Collaboration with the upstream developers resulted in patches that implemented more robust, endian-neutral data handling [104, 105]. Code in 729.abc and 735.gem5 also contained pointer dereferences and data operations that implicitly assumed a little-endian memory model. These sections were refactored to use endian-agnostic methods and explicit endianness checks, with fixes contributed back to the respective projects [106, 107].

Exposure of Latent Software Bugs. The porting process of 721.gcc to AIX uncovered a latent bug in the GCC compiler’s `tree-vrp` optimization pass specific to big-endian targets [108]. As backporting the upstream fix was infeasible for the software version used by the benchmark, the committee implemented a targeted workaround by disabling the problematic pass (`-fno-tree-vrp`) from one specific workload that was affected. This ensured consistent benchmark execution across all platforms without altering its fundamental behavior.

C. Operating Systems

The committee prioritized enabling CPU2026 to run across multiple operating systems, including Microsoft Windows running on both x86-64 and aarch64 platforms. Most of the applications considered for CPU2026 were developed for Unix-like systems, which means they had either limited or no prior support for running on Windows. A key aid to supporting Windows was to use MinGW (Minimalized GNU for Windows) [109] with gcc and gfortran compilers. Using MinGW helped diagnose whether issues were attributable to the Windows OS, to Windows compilers, or to a code dependency on GNU or POSIX functions.

Some applications needed little to no modification; others required substantial investigation and patches. For example, 721.gcc required changes in around 6000 distinct lines in a code base of over 4 million total lines. Bringing 735.gem5 to Windows/MSVC included splicing out vast chunks of the code base that were unexercised by the chosen workloads, an approach which greatly reduced the total porting effort.

The source code modifications fell into several common categories. A significant portion of the work involved refactoring file I/O and path handling to accommodate Windows-specific file system conventions. Another common task was resolving data model discrepancies, most notably by addressing differences in the size of the `long` data type (4 bytes on Windows vs. 8 bytes on most 64-bit Unix-like systems). Extensive changes were needed to resolve platform-specific dependencies, typically by substituting non-portable GNU/POSIX library functions with Windows-native equivalents and including the correct header files. In addition, some changes were needed to handle OS-specific constraints, such as maximum file path length and executable file size.

This validation process uncovered real computation bugs in the software, one which required consulting an astrodynamics

textbook written in 1971! [110]. Other issues that caused Windows-only errors were proactively upstreamed and accepted by the community [111, 112]. A code sequence with multiple virtual base pointers in 734.vpr exposed a memory size calculation error in LLVM, which was only seen in the Microsoft ABI [113].

These initiatives underscore a key benefit of cross-platform testing. Porting to less common architectures (big-endian), or operating systems (Windows, macOS, Android), rigorously tests the implicit assumptions made during development on more homogeneous platforms. The process not only hardens the benchmark code, making it more robust and portable, but also provides tangible benefits back to the open-source communities through upstreamed bug fixes. This feedback loop enhances the quality of both the SPEC CPU suite and the foundational applications upon which it is built.

D. IO Analysis and Reduction

The SPEC CPU benchmark suite is designed primarily to assess a processor’s computational performance and its interaction with the hierarchical memory subsystem. Introducing disk or network I/O into such benchmarks can lead to unpredictable delays that vary significantly across platforms, thereby obscuring the true scalability of the processor. Sources of variability include differences in storage device performance (e.g., NVMe, SSD, HDD), file system overhead, and network latency. When the CPU is forced to wait for I/O operations, overall utilization decreases, resulting in an inaccurate representation of computational capability. This issue is particularly pronounced in multi-threaded workloads, where threads can block waiting on I/O rather than fully exercising the processor’s computational resources [70].

A variety of Linux-based tools were employed to quantify I/O interactions and assess their impact on benchmark accuracy including `strace` [114] to monitor system call interactions with the kernel and `sar` [115] to capture historical I/O activity. Additionally, `emon` [116] was used to compare I/O bandwidth between 1-copy and 256-copy configurations. The analysis focused on quantifying the frequency and volume of I/O-related system calls to ensure that workloads remained fundamentally compute-bound, and that many-copy scaling was not impacted by increases in I/O activity.

Consistent with the suite’s long-standing character, several candidate workloads required optimization to bring their I/O behavior in line with (or better than) prior suites. Several mitigation techniques were applied across the benchmarks. To reduce I/O volume, input and output files were trimmed, and early or periodic writes were removed. At the code level, unbuffered output operations (e.g., `fprintf`) were replaced with buffered equivalents like `std::stringstream` to decimate the number of write system calls [117, 118]. Additionally, extraneous stream flushes were removed by replacing `std::endl` with the newline character `'\n'` [119], and inefficient, repeated `open/close` call sequences were consolidated [120]. These changes leverage the operating system’s ability to buffer writes within a memory page, thereby minimizing

the frequency of system calls and keeping the performance focus on the CPU and memory subsystems.

E. Memory Safety and Code Sanitization

As part of a broader commitment to delivering high-quality, robust software, a specific focus was placed on ensuring memory safety. Recent guidance from expert practitioners [121–123] as well as government agencies [124, 125], has made it an important aspect of modern software development. This industry-wide imperative is particularly relevant for the SPEC CPU suite due to its long-term, archival nature; in fact, modern analysis tools have identified out-of-bounds violations in CPU2006 [126] and CPU2017 [127, 128]. Thus, as benchmarks are frozen upon release and used for decades, there is a heightened responsibility to ensure their codebases are free from latent defects, particularly memory safety vulnerabilities.

To meet this responsibility, a comprehensive validation process was employed, utilizing a suite of software and hardware-assisted sanitization techniques. Each benchmark was tested with the Address Sanitizer (ASan) from both GCC and LLVM [129, 130] to detect issues like buffer overruns and use-after-free errors. The multithreaded SPECspeed benchmarks were then tested with Thread Sanitizer (TSan) [131] to identify data races. This process was augmented by hardware-accelerated validation using the ARM Memory Tagging Extension (MTE) available on AmpereOne® processors [132].

This multi-faceted approach proved effective, successfully identifying and enabling the correction of several previously unknown issues. For instance, MTE was instrumental in discovering memory safety defects in 767.nest [133] and 735.gem5 [134]. TSan then uncovered thread data races in 867.nest [135] and 837.gmsh [136]. All identified issues were patched with help from the community, with fixes contributed back to the respective upstream open-source projects. This exhaustive sanitization process provides high confidence for memory safety, at least within the scope of the code paths exercised by each benchmark’s workloads.

VI. REFERENCE SYSTEM

SPEC chooses a reference machine to normalize the performance and energy metrics used in the CPU benchmark suites. Each benchmark is run and measured on this machine to establish a reference time and energy for that benchmark. These values are then used in the SPEC ratio calculations to establish reportable scores.

TABLE II: SPEC CPU Reference Machines

CPU suite	System	CPU
CPU 89/92	DEC VAX-11/780	5 MHz DEC KA780
CPU 95	Sun SPARCstation 10/40	40 MHz SuperSPARC SM40
CPU 2000	Sun Ultra5_10	300 MHz UltraSPARC Iii
CPU 2006	Sun Ultra Enterprise 2	296 MHz UltraSPARC II
CPU 2017	Sun Fire V490	2.1 GHz UltraSPARC-IV+
CPU 2026	Lenovo TS HR330A	3.0 GHz Ampere eMAG 8180

The reference machine for CPU2026 is a historical Lenovo ThinkSystem HR330A [137] which uses the Ampere eMAG™ 8180 64-bit processor [138]. The eMAG processor, introduced in 2018, used the ARMv8 aarch64 ISA and supported up to 32 cores. Table II shows the reference machines used over time. One motivation for choosing older hardware is to ensure that resulting scores for modern machines will be above 1.0.

Note that when comparing any two systems measured with SPEC CPU (within the same suite version), their performance relative to each other would remain the same even if a different reference machine were used. This is a consequence of the math involved in calculating the individual and overall geomean metrics.

VII. ANALYSIS

Benchmark characterization has been used for decades to guide architectural design [139]. Many experiments were conducted on previous generations of SPEC CPU to correlate candidate behavior to contemporary application trends. These include studies conducted during development to discover hot function routines [71], to highlight challenges and insights derived from event-based analysis [140], to compare suite versions [141–143], or to perform statistical analyses [144]. Academic studies provide a memory-centric characterization of SPEC CPU2017, detailing memory footprints and bandwidth patterns [145], or evaluating memory hierarchy response [146]. Some even combined top-down analysis with energy metrics [56].

Prior SPEC CPU suites have served as a cornerstone for academic and industry research, enabling countless studies on workload characterization, architectural innovation, and performance modeling; the expectation is that CPU2026 will carry this legacy forward. Therefore, the analyses presented below are merely introductory, to showcase the kinds of data used by the committee for the process of benchmark selection.

A. PMC Characterization

Performance Monitoring Counters (PMCs) are registers built into modern CPUs that record low-level microarchitectural events during program execution, such as instructions commits, cache misses, and branch mispredictions. PMC characterization and Top-down Microarchitectural Analysis [147] is used to better understand the general behaviors and performance bottlenecks of the benchmark candidates. Detailed breakdowns of the bottlenecks can be found in the Appendix².

Taken together, instructions-per-cycle (IPC) and stall distributions offer a high-level perspective on the SPEC CPU2026 suites. The integer suites tend to exhibit more balanced frontend and backend bottlenecks, whereas floating-point suites are more consistently backend-bound. Some exceptions stand out, for example, 709.cactus shows notable frontend pressure despite being a floating-point workload, and compression oriented applications such as 777.zstd and 731.ascenc exhibit the highest fraction of cycles lost to speculation, consistent with

²Please see the extended edition of this paper, available on arXiv [66].

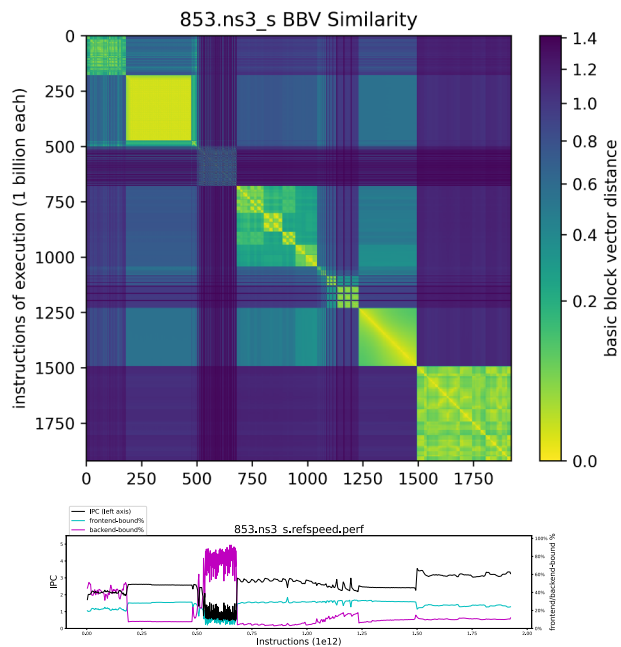


Fig. 1: Self-similarity recurrence plot alongside a performance plot collected from an AMD EPYC™ 9755. 853.ns3 is one of the few single-threaded SPECspeed benchmarks, which means it can accommodate this study. It consists of seven workloads which can be seen as seven major squares; the correlation is visually apparent between BBV recurrence regions and high level performance behaviors of the perf plot. The third square, starting at 500B instructions, is a workload that exhibits a large degree of DTLB misses, hence the spike in backend bottlenecks and lower IPC in that region. It is also the most dissimilar workload from the other six, as evidenced by the dark regions when comparing it to other workloads. The seventh workload, which starts at 1500B, is also dissimilar to the others.

the control-flow irregularity typical in data compression. These observations reflect the behavior of one particular system; other microarchitectures will showcase different bottlenecks.

B. BBV Recurrence Plots

One way to observe the internal behavior of a program is the analysis of its functional execution phases through Basic Block Vector (BBV) analysis, a method pioneered by the SimPoints toolkit [148] and available in Valgrind [149]. A basic block is a sequence of instructions with a single entry and exit point. A BBV captures the execution frequency of each basic block, indexed by the program counter of its entry instruction, over a fixed interval of execution (e.g., 10 million instructions). Running this over the entire benchmark results in a series of very high-dimensional vectors, each representing a snapshot of the program’s behavior. The similarity between any two execution intervals can be quantified by calculating the Euclidean distance between their corresponding BBVs; a smaller distance implies more similar behavior. Every BBV can be compared to every other BBV, which creates an $N \times N$ matrix. This matrix of distance can be visualized to characterize the program’s phase behavior across its entire run [150], resulting in the self-similarity plot as seen in Figure 1.

In the recurrence plot, each point (i, j) is colored based on the distance between the BBVs in interval i and interval j . This plot provides a qualitative assessment of a benchmark’s execution diversity. For workload selection, it helps identify redundancy; if two different inputs for a benchmark produced nearly identical regions in the recurrence plot, it signaled an opportunity to prune one in favor of a workload that exercised different code paths. Benchmarks with “peaky” function profiles tend to produce monotone yellow recurrence plots which can be spotted quickly. This style of visualization provides more insight than a textual function-level analysis alone, and offers another tool to assist in benchmark selection.

C. Perf Plots

To complement the BBV plots, performance plots can be generated from PMCs sampled over time. This helps examine how behavior evolves during execution. For each benchmark, these plots monitor IPC, frontend-bound percentage, and backend-bound percentage over time, allowing correlations among the three metrics, as well as fluctuations and phase changes within the individual workloads that constitute each benchmark. These time-series charts reveal finer-grained dynamics, highlighting periods of pipeline efficiency or stall dominance that vary across different workloads. Normalizing time-series to instructions allows overlaying with the BBV plots, offering a deeper analysis as seen in Figure 1. These pairings are offered in the Appendix³ for all the single-threaded benchmarks.

D. Parallelism

In SPECspeed, all 13 of the floating point benchmarks use parallelism, and 9 out of 13 integer benchmarks use parallelism. These are marked as MT in Table I. In total, these 22 parallel benchmarks use one of the following techniques: OpenMP 3.0, C++’s `std::thread`, Fortran’s `DO CONCURRENT`, or task-based process spawning. All of the SPECspeed threaded benchmarks are classified as strong scaling scenarios, as described in Table III. The SPECrate benchmarks are considered weak scaling, as the amount of work grows as the number of copies is increased.

TABLE III: Types of parallel software performance scaling in SPEC CPU

SPECrate® 2026	SPECspeed® 2026
Weak Scaling	Strong Scaling
Workload size increases with copies	Workload size remains constant
Goal is to maintain a constant time to complete the tasks as the workload size grows	Goal is to decrease the total time to complete the fixed-size workload by splitting it amongst processors
Gustafson’s Law: speedup based on the workload size scaling up to match the number of processors [151]	Amdahl’s Law: speedup is limited by the portions of the program that cannot be parallelized [152]

In addition to language based parallelism, for the first time SPEC CPU offers two benchmarks with task-based parallelism: 821.gcc and 823.llvm. These are based on the two

most popular open source compilers, and in both of these benchmarks, thousands of unique command lines are invoked to build multiple input source files. These benchmarks mimic the way ‘make -j N’ runs in the field; each command line spawns a new compiler process, which keeps N cores active until the large pile of work is completed.

VIII. RRR - HETEROGENEOUS SCHEDULE

CPU2026 introduces a new heterogeneous style of running benchmarks in multi-copy mode called Rolling Round-Robin, or RRR. Here we explain the motivation for this run style of multi-programmed workloads and a description of the methodology. RRR is in exhibition as the scoring methodology is not well-established; this is an open call to assist with its continuing evaluation.

SPEC CPU’s primary multi-copy benchmark, SPECrate, stresses systems with a homogeneous load, with each copy running the same component benchmark simultaneously. This single-program, homogeneous capacity methodology has been foundational to SPEC CPU since 1992 [153]. In today’s CPUs with hundreds of cores, this method can expose system corner cases, yet modern server systems do not usually operate under homogeneous loads. Multi-tenant systems operate with VMs running all manners of workloads simultaneously, hence the motivation for a heterogeneous style of benchmarking.

The increasing complexity and heterogeneity of modern multicore SoCs, particularly those deployed in cloud and data center environments for AI agentic workflows, necessitate robust methodologies for multiprogrammed workload characterization and performance evaluation. Single-program homogeneous runs cannot expose intricate cross-process interactions, heterogeneous resource contention, and scheduling dynamics that are prevalent in contemporary multiprogrammed systems. A challenge identified in the literature is the lack of a standardized approach to generate multiprogrammed benchmarks. Researchers frequently resort to custom-crafted benchmark mixtures and ad-hoc scheduling policies [154–159], hindering direct comparison across studies and limiting the generalization of architectural and software optimizations. This problem extends from defining benchmark composition and workload sampling methods [160, 161] to the selection of appropriate performance and fairness metrics [162–165]. Similar work [166] highlights the importance of distinguishing between sample imbalance (differences in standalone run-times) and schedule imbalance (asymmetric contention), yet many ad-hoc methodologies still introduce both. This fragmentation underscores a need for a standardized and reproducible methodology for evaluating multiprogrammed performance on heterogeneous multicore systems.

To address this gap, CPU2026 introduces the Rolling Round-Robin benchmarking mode [167]. RRR offers a standardized, deterministic schedule and repeatable method for utilizing the existing intrate and fprate suites as multiprogrammed benchmarks. For a suite comprising N benchmarks (e.g., 14 for intrate) running on M cores, the RRR methodology operates as follows: Each of M cores executes all N benchmarks

³Please see the extended edition of this paper, available on arXiv [66].

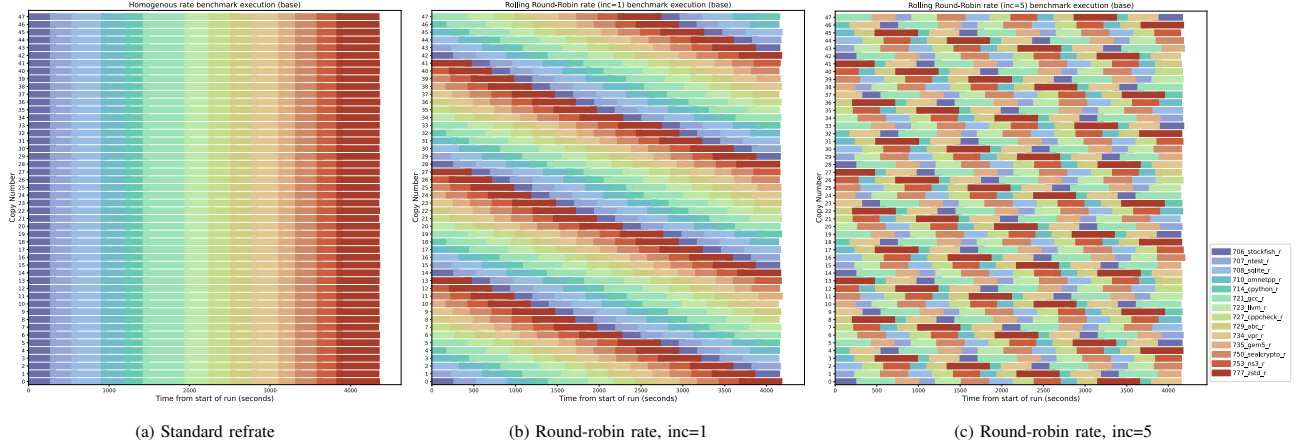


Fig. 2: Time plots showing all the integer rate benchmarks executing on schedules for homogenous (refrate) and heterogeneous (rrrate) on 48 copies. In all cases, each copy runs every benchmark once. The harness interface allows using a subset of benchmarks as well, allowing the user to craft their own scenario to run using the round-robin scheduler. The “inc” parameter tells the scheduler how to increment through the roster of benchmarks, allowing for some customization in the heterogeneous scheduling. Plots were generated with the `rate_timeplot.py` script found in the `scripts.misc` directory bundled with the CPU2026 distribution.

sequentially, in a predetermined fixed order, rotating through the benchmarks. For example, if Core 0 starts with Benchmark A, Core 1 would start with Benchmark B, and so on, cyclically through the entire benchmark roster. Each core continues to run its assigned schedule of benchmarks until all N benchmarks have completed on all M cores. This design ensures that every benchmark runs on every core, providing a uniform exposure while allowing for the exploration of diverse contention scenarios in a controlled and repeatable fashion. This methodical approach inherently eliminates sample imbalance, as each benchmark is guaranteed to run to completion an equal number of iterations across the system, ensuring identical instruction counts for each benchmark and each core, from the perspective of user-level CPU execution. Figure 2 compares three different styles of schedule methods, from runs on an AmpereOne® system executing 48 copies.

The RRR methodology provides a much-needed common ground for rigorous research into multicore performance. Since the benchmarks have been hardened through portability (§V), there is no worry of syscalls or IO or similar wrenches impacting the schedule. This allows the focus to remain on user level CPU performance under heterogenous load. By standardizing the workload construction, RRR enables direct and fair comparisons of architectural innovations, OS scheduling policies [168, 169], and resource partitioning schemes [170, 171]. It allows multiprogramming studies to move beyond methods that substitute manual kernel isolation for full complex applications [172, 173]. RRR enables benchmarks to interact with the microarchitecture and each other in ways more representative of their complete execution profiles, preserving “unwanted cross-effects” that might otherwise be overlooked [172]. While RRR standardizes the generation of one style of multiprogrammed workloads, the research community continues to debate the optimal metrics for evaluating such

systems (e.g., cumulative IPC, average throughput, harmonic mean, fairness indices) [162–165]. RRR provides the raw, standardized execution data upon which various metrics can be proposed and compared, fostering further discussion on the most appropriate performance and fairness indicators for complex heterogeneous systems. By offering a robust and deterministic framework to control heterogenous load, the SPEC CPU committee believes RRR will significantly enhance the rigor and comparability of future multicore research.

IX. CONCLUSION

The physicist Lord Kelvin famously stated, “If you can not measure it, you can not improve it.” This maxim has served as the unspoken charter for the field of computer architecture, where progress is inextricably linked to the ability to perform fair, repeatable, and relevant measurements. For over three decades, the SPEC CPU suite has been the industry’s primary instrument for this purpose. This paper has detailed the creation of CPU2026, the seventh version of this essential tool, reaffirming the enduring importance of general-purpose compute performance in an era of increasing specialization. By providing a transparent account of the methodology, we have demonstrated that the suite is not a synthetic construct, but a carefully curated collection of real-world applications, hardened into a robust and portable benchmark.

This paper has chronicled the principled and methodical process behind the development of CPU2026. We have shown how a symbiotic relationship with the open-source community, formalized through a multi-year search program, yielded a suite with an expanded set of benchmarks exhibiting novel microarchitectural profiles not seen in prior suites. Key contributions were detailed, including the landmark introduction of multithreaded integer workloads and the new Rolling Round-Robin Rate (RRR) methodology for characterizing heterogeneous throughput on multicore processors. Furthermore, we

outlined the extensive engineering discipline required to create a long-lived suite, from meticulous code hardening for ISO compliance and portability across diverse platforms like big-endian systems and Microsoft Windows, to a modern commitment to memory safety validated by a battery of software and hardware sanitizers.

In his lecture *Technology and Courage*, Ivan Sutherland spoke of the courage required to embark on a new and uncertain endeavor [174]. In that light, this work honors the courage of our SPEC predecessors who, despite being commercial competitors, came together to pursue a shared vision of microprocessor benchmarks that are fair, comparable, and representative [2]. The current generation of stewards has continued to manifest this vision with the release of SPEC CPU2026. The suite is now entrusted to the community—to the architects, compiler developers, software experts, and researchers who will use it to test their own courageous ideas. The baton is now passed to you, the assiduous reader, to build upon this foundation and continue the march toward ever stronger computing benchmarks. You are the Next Generation.

ACKNOWLEDGEMENT

The creation of SPEC CPU®2026 was a significant collaborative effort, and the authors wish to express their profound gratitude to the individuals whose leadership and contributions were essential. Special thanks are extended to the four individuals who served as committee chair over the course of the development process: Jeff Reilly, James Bucek, Van Smith, and Frédérique Silber-Chaussy. Their guidance was instrumental in steering a diverse committee toward a common goal and successfully delivering this benchmark suite. Deepest gratitude is extended to John Henning, the committee secretary, for his overall mentorship drawn from his encyclopedic knowledge of SPEC CPU history, and for scrutinizing the suite with an eye for licensing. We are also grateful to Cloyce Spradling, the release manager, for his meticulous work in integrating the components and managing the final publication. We thank Ronen Zohar for his expertise on C/C++ language standards and complex compiler behaviors, and Sunil Vijay Sathe for sharing his specialized knowledge of HPC workloads. Finally, we acknowledge Ruihao Li, Neeraja Yadwadkar, and Lizy John for providing the dendrogram analysis that served as a vital tool in the benchmark selection process.

This suite would not have been possible without the collective dedication and expertise of the entire SPEC CPU committee, supporting contributors, and the participants of the CPUv8 benchmark search program.

REFERENCES

- [1] D. Patterson, “For Better or Worse, Benchmarks Shape a Field: Technical Perspective,” *Commun. ACM*, vol. 55, no. 7, p. 104, Jul. 2012, ISSN: 0001-0782. DOI: [10.1145/2209249.2209271](https://doi.org/10.1145/2209249.2209271).
- [2] K. M. Dixit, “Overview of the SPEC Benchmarks,” in *The Benchmark Handbook*, 1993. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6617287>.
- [3] J. L. Henning, “SPEC CPU2000: Measuring CPU Performance in the New Millennium,” *Computer*, vol. 33, no. 7, 28–35, Jul. 2000, ISSN: 0018-9162. DOI: [10.1109/2.869367](https://doi.org/10.1109/2.869367).

- [4] J. L. Henning, “SPEC CPU2006 benchmark descriptions,” *SIGARCH Comput. Archit. News*, 2006. DOI: [10.1145/1186736.1186737](https://doi.org/10.1145/1186736.1186737).
- [5] J. Bucek, K.-D. Lange, and J. v. Kistowski, “SPEC CPU2017: Next-Generation Compute Benchmark,” in *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE ’18, Berlin, Germany: ACM, 2018, 41–42, ISBN: 9781450356299. DOI: [10.1145/3185768.3185771](https://doi.org/10.1145/3185768.3185771).
- [6] R. Munafo, *The SPEC Benchmarks*, 2025. [Online]. Available: <https://www.mrob.com/pub/comp/benchmarks/spec.html>.
- [7] A. Danowitz, K. Kelley, J. Mao, J. P. Stevenson, and M. Horowitz, “CPU DB: Recording Microprocessor History,” *Commun. ACM*, vol. 55, no. 4, 55–63, Apr. 2012, ISSN: 0001-0782. DOI: [10.1145/2133806.2133822](https://doi.org/10.1145/2133806.2133822).
- [8] P. Tüma, *Long Term Stability Observations*, DaCapo Benchmark, 2024. [Online]. Available: <https://github.com/dacapobench/dacapobench/issues/269>.
- [9] P. Kessler, *Variance in DaCapo Benchmark*, DaCapo Benchmark, 2024. [Online]. Available: <https://github.com/dacapobench/dacapobench/issues/302#issuecomment-2469456922>.
- [10] A. M. Yang, *Large Variance in Exec Time for scala-doku*, Renaissance Benchmark, 2019. [Online]. Available: <https://github.com/renaissance-benchmarks/renaissance/issues/175>.
- [11] L. Bulej, *Renaissance 0.16*, Renaissance Benchmark, 2024. [Online]. Available: <https://renaissance.dev/2024/11/22/renaissance-0-16-0.html>.
- [12] H. Vandierendonck and K. D. Bosschere, “Many Benchmarks Stress the Same Bottlenecks,” in *Workshop on Computer Architecture Evaluation Using Commercial Workloads*, 2004. [Online]. Available: <https://api.semanticscholar.org/CorpusID:11877774>.
- [13] Y. Wang, S. Furman, N. Hardy, M. Ellis, G. Back, Y. Hong, and K. Cameron, *A Detailed Historical and Statistical Analysis of the Influence of Hardware Artifacts on SPEC Integer Benchmark Performance*, 2024. arXiv: [2401.16690](https://arxiv.org/abs/2401.16690) [cs.CY].
- [14] C. Wang, L. Wang, W. Gao, F. Fan, Y. Su, Y. Zhou, Y. Yang, and J. Zhan, *Achieving Consistent and Comparable CPU Evaluation*, 2025. arXiv: [2411.08494](https://arxiv.org/abs/2411.08494) [cs.PF].
- [15] The Tukaani Project, *XZ Utils*. [Online]. Available: https://en.wikipedia.org/wiki/XZ_Utils.
- [16] T. Romstad, M. Costalba, and J. Kiiski, *Stockfish: Strong open-source chess engine*. [Online]. Available: <https://stockfishchess.org/>.
- [17] C. Welty and V. Petric, *NTest: A Strong Othello Program*. [Online]. Available: <https://github.com/vladpetric/ntest>.
- [18] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, “SQLite: past, present, and future,” *Proceedings of the VLDB Endowment*, vol. 15, no. 12, 2022. DOI: [10.14778/3554821.3554842](https://doi.org/10.14778/3554821.3554842). [Online]. Available: <https://sqlite.org/about.html>.
- [19] A. Varga and R. Hornig, “An Overview of the OMNeT++ Simulation Environment,” in *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems & Workshops*, ser. Simutools ’08, Marseille, France: ICST, 2008, ISBN: 9789639799202. DOI: [10.4108/ICST.SIMUTOOLS2008.3027](https://doi.org/10.4108/ICST.SIMUTOOLS2008.3027). [Online]. Available: www.omnetpp.org.
- [20] G. Rossum, “Python reference manual,” CWI (Centre for Mathematics and Computer Science), Netherlands, Tech. Rep., 1995. [Online]. Available: <https://ir.cwi.nl/pub/5008>.
- [21] J. Coalson, E. de Castro Lopo, and M. van Beurden, *flac: Free Lossless Audio Codec*. [Online]. Available: <https://xiph.org/flac>.
- [22] R. Stallman, *GCC, the GNU Compiler Collection*. [Online]. Available: <https://gcc.gnu.org>.
- [23] C. Lattner and V. Adve, “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation,” in *International Symposium on Code Generation and Optimization*, 2004. CGO 2004., 2004, pp. 75–86. DOI: [10.1109/CGO.2004.1281665](https://doi.org/10.1109/CGO.2004.1281665). [Online]. Available: www.llvm.org.
- [24] D. Marjamäki, *Cppcheck: a tool for static C/C++ code analysis*. [Online]. Available: www.cppcheck.com.
- [25] R. Brayton and A. Mishchenko, “ABC: An Academic Industrial-Strength Verification Tool,” in *Computer Aided Verification*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 24–40, ISBN: 978-3-642-14295-6. [Online]. Available: <https://people.eecs.berkeley.edu/~alanmi/abc/>.
- [26] M. A. Elgammal, A. Mohaghegh, S. G. Shahrour, F. Mahmoudi, F. Koşar, K. Talaei, J. Fife, D. Khadivi, K. Murray, A. Boutros,

- K. B. Kent, J. Goeders, and V. Betz, "VTR 9: Open-Source CAD for Fabric and Beyond FPGA Architecture Exploration," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 18, Aug. 2025, ISSN: 1936-7406. DOI: [10.1145/3734798](https://doi.org/10.1145/3734798). [Online]. Available: <https://verilogtorouting.org>.
- [27] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, S. Bharadwaj, G. Black, G. Bloom, B. R. Bruce, D. R. Carvalho, J. Castrillón, L. Chen, N. Derumigny, S. Diestelhorst, W. Elsasser, M. Fariborz, A. F. Farahani, P. Fotouhi, R. Gambord, J. Gandhi, D. Gope, T. Grass, B. Hanindhito, A. Hansson, S. Haria, A. Harris, T. Hayes, A. Herrera, M. Horsnell, S. A. R. Jafri, R. Jagtap, H. Jang, R. Jeyapaul, T. M. Jones, M. Jung, S. Kannoth, H. Khaleghzadeh, Y. Kodama, T. Krishna, T. Marinelli, C. Menard, A. Mondelli, T. Mück, O. Naji, K. Nathella, H. Nguyen, N. Nikoleris, L. E. Olson, M. S. Orr, B. Pham, P. Prieto, T. Reddy, A. Roelke, M. Samani, A. Sandberg, J. Setoain, B. Shingarov, M. D. Sinclair, T. Ta, R. Thakur, G. Travaglini, M. Upton, N. Vaish, I. Vougioukas, Z. Wang, N. Wehn, C. Weis, D. A. Wood, H. Yoon, and É. F. Zulian, "The gem5 Simulator: Version 20.0+," 2020. arXiv: [2007.03152](https://arxiv.org/abs/2007.03152). [Online]. Available: www.gem5.org.
- [28] B. Buchfink, K. Reuter, and H.-G. Drost, "Sensitive protein alignments at tree-of-life scale using DIAMOND," in *Nature Methods*, vol. 18, 2021. DOI: [10.1038/s41592-021-01101-x](https://doi.org/10.1038/s41592-021-01101-x). [Online]. Available: <https://github.com/bbuchfink/diamond>.
- [29] N. Nethercote, P. J. Stuckey, R. Becket, S. Brand, G. J. Duck, and G. Tack, "MiniZinc: Towards a Standard CP Modelling Language," in *Principles and Practice of Constraint Programming – CP 2007*, Berlin, Heidelberg: Springer, 2007, pp. 529–543, ISBN: 978-3-540-74970-7. [Online]. Available: www.minizinc.org.
- [30] H. Chen, K. Laine, and R. Player, "Simple Encrypted Arithmetic Library - SEAL v2.1," in *Financial Cryptography and Data Security*, Cham: Springer International Publishing, 2017, pp. 3–18, ISBN: 978-3-319-70278-0. DOI: [10.1007/978-3-319-70278-0_1](https://doi.org/10.1007/978-3-319-70278-0_1). [Online]. Available: <https://github.com/Microsoft/SEAL>.
- [31] G. F. Riley and T. R. Henderson, "The ns-3 Network Simulator," in *Modeling and Tools for Network Simulation*. Berlin, Heidelberg: Springer, 2010, pp. 15–34, ISBN: 978-3-642-12331-3. DOI: [10.1007/978-3-642-12331-3_2](https://doi.org/10.1007/978-3-642-12331-3_2). [Online]. Available: www.nsnam.org.
- [32] R. C. Murphy, K. B. Wheeler, B. W. Barrett, and J. A. Ang, "Introducing the Graph 500," *Cray Users Group (CUG)*, vol. 19, no. 45-74, p. 22, 2010. [Online]. Available: <https://graph500.org>.
- [33] ZStd Community, *ZStd, the C reference implementation for the ZStandard lossless compression algorithm*. [Online]. Available: <https://facebook.github.io/zstd>.
- [34] C. Downs, J. A. Linker, R. M. Caplan, E. I. Mason, P. Riley, R. Davidson, A. Reyes, E. Palmerio, R. Lionello, J. Turtle, M. Ben-Nun, M. M. Stulajter, V. S. Titov, T. Török, L. A. Upton, R. Attie, B. K. Jha, C. N. Arge, C. J. Henney, G. Valori, H. Strecker, D. Calchetti, D. Germerott, J. Hinzberger, D. O. Suárez, J. B. Rodríguez, S. K. Solanki, X. Cheng, and S. Wu, "A Near-real-time Data-assimilative Model of the Solar Corona," *Science*, vol. 388, no. 6753, pp. 1306–1310, 2025. DOI: [10.1126/science.adq0872](https://doi.org/10.1126/science.adq0872).
- [35] D. Guerrero, R. M. Cabezon, J.-G. Piccinali, A. Cavelan, F. M. Ciorba, D. Imbert, L. Mayer, and D. Reed, "Towards a Mini-App for Smoothed Particle Hydrodynamics at Exascale," in *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, 2018, pp. 607–614. DOI: [10.1109/CLUSTER.2018.00077](https://doi.org/10.1109/CLUSTER.2018.00077).
- [36] T. Goodale, G. Allen, G. Lanfermann, J. Masso, T. Radke, H. Seidel, and J. Shalf, "The Cactus Framework and Toolkit: Design and Applications," in *VECPA Vector and Parallel Processing R'2002, 5th International Conference, Springer (2003)*, vol. 2565, Jun. 2002, ISBN: 978-3-540-00852-1. DOI: [10.1007/3-540-36569-9_13](https://doi.org/10.1007/3-540-36569-9_13). [Online]. Available: www.cactuscode.org.
- [37] S. McIntosh-Smith, M. Martineau, T. Deakin, G. Pawelczak, W. Gaudin, P. Garrett, W. Liu, R. Smedley-Stevenson, and D. Beckingsale, "TeaLeaf: A Mini-Application to Enable Design-Space Explorations for Iterative Sparse Linear Solvers," in *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, 2017, pp. 842–849. DOI: [10.1109/CLUSTER.2017.105](https://doi.org/10.1109/CLUSTER.2017.105).
- [38] T. J. Macke and D. A. Case, "Modeling Unusual Nucleic Acid Structures," in *Molecular Modeling of Nucleic Acids*. American Chemical Society, 1997, ch. 24, pp. 379–393. DOI: [10.1021/bk-1998-0682.ch024](https://doi.org/10.1021/bk-1998-0682.ch024). [Online]. Available: <https://casegroup.rutgers.edu>.
- [39] M. A. Heroux, "Mantevo 3.0 Overview," Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), Tech. Rep., 2015. [Online]. Available: <https://mantevo.github.io/applications.html>.
- [40] B. Maronga, S. Banzhaf, C. Burmeister, T. Esch, R. Forkel, D. Fröhlich, V. Fuka, K. F. Gehrke, J. Geletić, S. Giersch, T. Gronemeier, G. Groß, W. Heldens, A. Hellsten, F. Hoffmann, A. Inagaki, E. Kadasch, F. Kanani-Sühring, K. Ketelsen, B. A. Khan, C. Knigge, H. Knoop, P. Krč, M. Kurppa, H. Maamari, A. Matzarakis, M. Mauder, M. Pallasch, D. Pavlik, J. Pfaffert, J. Resler, S. Rissmann, E. Russo, M. Salim, M. Schrepf, J. Schwenkel, G. Seckmeyer, S. Schubert, M. Sühring, R. von Tils, L. Vollmer, S. Ward, B. Witha, H. Wurps, J. Zeidler, and S. Raasch, "Overview of the PALM model system 6.0," *Geoscientific Model Development*, vol. 13, no. 3, pp. 1335–1372, 2020. DOI: [10.5194/gmd-13-1335-2020](https://doi.org/10.5194/gmd-13-1335-2020). [Online]. Available: <https://docs.palm-model.com/25.04/>.
- [41] P. Harris, *The ARM ASTC Encoder, a compressor for the Adaptive Scalable Texture Compression data format*. [Online]. Available: <https://github.com/ARM-software/astc-encoder>.
- [42] D. Walker, M. Dolan, and P. Hodoul, "The ASWF Takes Open-ColorIO to the Next Level," in *Proceedings of the 2020 Digital Production Symposium*, ser. DigiPro '20, Virtual Event, USA: ACM, 2020, ISBN: 9781450380348. DOI: [10.1145/3403736.3403942](https://doi.org/10.1145/3403736.3403942). [Online]. Available: www.opencolorio.org.
- [43] C. Geuzaine and J.-F. Remacle, "GMSH: A 3-D finite element mesh generator with built-in pre- and post-processing facilities," *International Journal for Numerical Methods in Engineering*, vol. 79, no. 11, pp. 1309–1331, 2009. DOI: [10.1002/nme.2579](https://doi.org/10.1002/nme.2579). [Online]. Available: www.gmsh.info.
- [44] J. Berndt, "JSBSim: An Open Source Flight Dynamics Model in C++," in *AIAA Modeling and Simulation Technologies Conference and Exhibit*. AIAA Journal, 2012. DOI: [10.2514/6.2004-4923](https://doi.org/10.2514/6.2004-4923). [Online]. Available: <https://github.com/JSBSim-Team/jsbsim>.
- [45] U. Andersson, M. Qiu, and Z. Zhang, "Parallel power computation for photonic crystal devices," *Methods and Applications of Analysis*, vol. 13, pp. 149–156, Jul. 2006. DOI: [10.4310/MAA.2006.v13.n2.a3](https://doi.org/10.4310/MAA.2006.v13.n2.a3).
- [46] J. C. Phillips, R. Braun, W. Wang, J. Gumbart, E. Tajkhorshid, E. Villa, C. Chipot, R. D. Skeel, L. Kalé, and K. Schulten, "Scalable molecular dynamics with namd," *Journal of Computational Chemistry*, vol. 26, no. 16, pp. 1781–1802, 2005. DOI: [10.1002/jcc.20289](https://doi.org/10.1002/jcc.20289). [Online]. Available: www.ks.uiuc.edu/Research/namd.
- [47] A. F. Shchepetkin and J. C. McWilliams, "The Regional Oceanic Modeling System (ROMS): a Split-explicit, Free-surface, Topography-following-coordinate Oceanic Model," *Ocean Modelling*, vol. 9, no. 4, 2005, ISSN: 1463-5003. DOI: [10.1016/j.ocemod.2004.08.002](https://doi.org/10.1016/j.ocemod.2004.08.002). [Online]. Available: www.myroms.org.
- [48] M. Kronbichler, D. Sashko, and P. Munch, *Enhancing data locality of the conjugate gradient method for high-order matrix-free finite-element implementations*, 2022. arXiv: [2205.08909](https://arxiv.org/abs/2205.08909) [cs.LG]. [Online]. Available: <https://github.com/kronbichler/spec-femflow>.
- [49] H. E. Plesser, M. Diesmann, M.-O. Gewaltig, and A. Morrison, "NEST: The Neural Simulation Tool," in *Encyclopedia of Computational Neuroscience*. New York, NY: Springer New York, 2022, pp. 2187–2189, ISBN: 978-1-0716-1006-0. DOI: [10.1007/978-1-0716-1006-0_258](https://doi.org/10.1007/978-1-0716-1006-0_258). [Online]. Available: <https://nest-initiative.org>.
- [50] M. Junczys-Dowmunt, R. Grundkiewicz, T. Dwojak, H. Hoang, K. Heafield, T. Neckermann, F. Seide, U. Germann, A. F. Aji, N. Bogoychev, A. F. T. Martins, and A. Birch, "Marian: Fast Neural Machine Translation in C++," 2018. arXiv: [1804.00344](https://arxiv.org/abs/1804.00344). [Online]. Available: <https://marian-nmt.github.io>.
- [51] T. Pohl, M. Kowarschik, J. Wilke, K. Iglberger, and U. Rüde, "Optimization And Profiling Of The Cache Performance Of Parallel Lattice Boltzmann Codes," *Parallel Processing Letters*, vol. 13, pp. 549–560, Dec. 2003. DOI: [10.1142/S0212962603001501](https://doi.org/10.1142/S0212962603001501).
- [52] J. R. Tramm, A. R. Siegel, B. Forget, and C. Josey, "Performance Analysis of a Reduced Data Movement Algorithm for Neutron Cross Section Data in Monte Carlo Simulations," in *Solving Software Challenges for Exascale*, Springer International Publishing, 2015, pp. 39–56, ISBN: 978-3-319-15976-8. DOI: [10.1007/978-3-319-15976-8_3](https://doi.org/10.1007/978-3-319-15976-8_3). [Online]. Available: <https://github.com/ANL-CESAR>.
- [53] R. S. Machado, R. B. Almeida, A. D. Jardim, A. M. Pernas, A. C. Yamin, and G. G. H. Cavalheiro, "Comparing Performance of C Compilers Optimizations on Different Multicore Architectures," in *2017 International Symposium on Computer Architecture and High*

- Performance Computing Workshops (SBAC-PADW)*, 2017, pp. 25–30. DOI: [10.1109/SBAC-PADW.2017.13](https://doi.org/10.1109/SBAC-PADW.2017.13).
- [54] A. Almomany, A. Alquraan, and L. Balachandran, “GCC vs. ICC comparison using PARSEC Benchmarks,” *International Journal of Innovative Technology and Exploring Engineering*, vol. 4, pp. 76–82, Dec. 2014. [Online]. Available: www.researchgate.net/publication/367255416_GCC_vs_ICC_comparison_using_PARSEC_Benchmarks.
 - [55] K. Halbiniak, R. Wyrzykowski, L. Szustak, A. Kulawik, N. Meyer, and P. Gepner, “Performance Exploration of Various C/C++ Compilers for AMD EPYC Processors in Numerical Modeling of Solidification,” *Advances in Engineering Software*, vol. 166, p. 103 078, 2022, ISSN: 0965-9978. DOI: [10.1016/j.advengsoft.2021.103078](https://doi.org/10.1016/j.advengsoft.2021.103078).
 - [56] R. Hebbar S R and A. Milenković, “SPEC CPU2017: Performance, Event, and Energy Characterization on the Core i7-8700K,” in *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE ’19, Mumbai, India: ACM, 2019, 111–118, ISBN: 9781450362399. DOI: [10.1145/3297663.3310314](https://doi.org/10.1145/3297663.3310314).
 - [57] SPEC, *SPEC CPU® v8 Benchmark Search Program*, 2020. [Online]. Available: www.spec.org/cpu/cpuv8.
 - [58] SPEC, *Step 3 Rules of the CPUv8 Benchmark Search Program*, 2020. [Online]. Available: www.spec.org/cpu/cpuv8/step3.
 - [59] ACM, *NAMD: First ACM Gordon Bell Special Prize for High Performance Computing-Based COVID-19 Research Awarded*, 2020. [Online]. Available: www.acm.org/media-center/2020/november/gordon-bell-special-prize-covid-research-2020.
 - [60] Community, *JSBSim for SPEC CPU v8*, JSBSim, 2023. [Online]. Available: <https://github.com/JSBSim-Team/jsbsim/issues/834>.
 - [61] Community, *NEST for SPEC CPU v8*, NEST, 2024. [Online]. Available: <https://github.com/nest/nest-simulator/issues/3217>.
 - [62] Academy of Motion Picture Arts and Sciences, *2013 Sci-Tech Awards: Jeremy Selan for OCIO*, YouTube, 2014. [Online]. Available: www.youtube.com/watch?v=PMTSbvdVxnE.
 - [63] SPEC, *CPU®2026 Licenses*. [Online]. Available: www.spec.org/cpu2026/Docs/licenses.html.
 - [64] L. Eeckhout, H. Vandierendonck, and K. De Bosschere, “Workload Design: selecting representative program-input pairs,” in *Proceedings of International Conference on Parallel Architectures and Compilation Techniques*, 2002, pp. 83–94. DOI: [10.1109/PACT.2002.1106006](https://doi.org/10.1109/PACT.2002.1106006).
 - [65] T. Bartz-Beielstein, C. Doerr, D. van den Berg, J. Bossek, S. Chandrasekaran, T. Eftimov, A. Fischbach, P. Kerschke, W. L. Cava, M. Lopez-Ibanez, K. M. Malan, J. H. Moore, B. Naujoks, P. Orzechowski, V. Volz, M. Wagner, and T. Weise, *Benchmarking in Optimization: Best Practice and Open Issues*, 2020. arXiv: [2007.03488](https://arxiv.org/abs/2007.03488) [cs.NE].
 - [66] M. Madhav, A. Lee, A. Mejia, B. Moore, C. Soppadandi, C. Cambly, C. Müllner, D. Bowers, D. Reiner, D. Bakhvalov, D. Zhao, D. Voth, F. Xue, F. Silber-Chaussumier, J. Bucek, J. Southern, J. Liu, J. Himer, J. Henning, K. Smith, K. Yang, K. Kashyap, M. Guy, M. Colgrove, M. Berg, P. Battini, P. Joshi, R. Prasad, S. Bhattacharya, S. Caculo, S. Reimbold, S. Iyengar, V. Smith, and Z. Todorovski, “SPEC CPU: The Next Generation,” in *Proceedings of the 53rd Annual International Symposium on Computer Architecture*, ser. ISCA ’26, Raleigh, NC, USA: IEEE, 2026. arXiv: [2605.01575](https://arxiv.org/abs/2605.01575) [cs.PF].
 - [67] J. N. Amaral, E. Borin, D. R. Ashley, C. Benedicto, E. Colp, J. H. S. Hoffmann, M. Karpoff, E. Ochoa, M. Redshaw, and R. E. Rodrigues, “The Alberta Workloads for the SPEC CPU 2017 Benchmark Suite,” in *IEEE ISPASS*, 2018, pp. 159–168. DOI: [10.1109/ISPASS.2018.00029](https://doi.org/10.1109/ISPASS.2018.00029). [Online]. Available: <https://webdocs.cs.ualberta.ca/~amaral/AlbertaWorkloadsForSPEC2017/>.
 - [68] O. Alonso and K. Church, “Evaluating the Evaluations: A Perspective on Benchmarks,” *SIGIR Forum*, vol. 58, no. 2, 1–27, Mar. 2025, ISSN: 0163-5840. DOI: [10.1145/3722449.3722467](https://doi.org/10.1145/3722449.3722467).
 - [69] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy, and M. J. Kochenderfer, “BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices,” in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS ’24, Vancouver, BC, Canada: Curran Associates Inc., 2024, ISBN: 9798331314385. arXiv: [2411.12990](https://arxiv.org/abs/2411.12990) [cs.AI].
 - [70] D. Ye, J. Ray, and D. Kaeli, “Characterization of File I/O Activity for SPEC CPU2006,” *SIGARCH Comput. Archit. News*, vol. 35, no. 1, 112–117, Mar. 2007, ISSN: 0163-5964. DOI: [10.1145/1241601.1241622](https://doi.org/10.1145/1241601.1241622).
 - [71] R. P. Weicker and J. L. Henning, “Subroutine Profiling Results for the CPU2006 Benchmarks,” *SIGARCH Comput. Archit. News*, vol. 35, no. 1, 102–111, Mar. 2007, ISSN: 0163-5964. DOI: [10.1145/1241601.1241621](https://doi.org/10.1145/1241601.1241621).
 - [72] J. Wittich, *Ampere Strategy and Roadmap Update*, 2024. [Online]. Available: www.youtube.com/watch?v=cYrT2ohcykk&t=1404s.
 - [73] D. Brown, *AWS re:Invent 2023 - Compute innovation for any application, anywhere*, 2023. [Online]. Available: www.youtube.com/watch?v=dxm93_qgRzk&t=2210s.
 - [74] J. L. Henning, “SPEC CPU Suite Growth: an Historical Perspective,” *SIGARCH Comput. Archit. News*, vol. 35, no. 1, 65–68, Mar. 2007, ISSN: 0163-5964. DOI: [10.1145/1241601.1241615](https://doi.org/10.1145/1241601.1241615).
 - [75] R. Panda, S. Song, J. Dean, and L. K. John, “Wait of a Decade: Did SPEC CPU 2017 Broaden the Performance Horizon?” In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018, pp. 271–282. DOI: [10.1109/HPCA.2018.00032](https://doi.org/10.1109/HPCA.2018.00032).
 - [76] W. Su, A. Dhanotia, C. Torres, J. Gandhi, N. Gholkar, S. Kanaujia, M. Naumov, K. Subramanian, V. Andrei, Y. Yuan, and C. Tang, “DCPerf: An Open-Source, Battle-Tested Performance Benchmark Suite for Datacenter Workloads,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25, Tokyo, Japan: ACM, 2025, 1717–1730, ISBN: 9798400712616. DOI: [10.1145/3695053.3731411](https://doi.org/10.1145/3695053.3731411).
 - [77] D. Citron, “MisSPECulation: Partial and Misleading Use of SPEC CPU2000 in Computer Architecture Conferences,” in *Proceedings of the 30th Annual International Symposium on Computer Architecture*, San Diego, California: ACM, 2003, 52–61, ISBN: 0769519458. DOI: [10.1145/859618.859625](https://doi.org/10.1145/859618.859625).
 - [78] IEEE Micro staff, “The Use and Abuse of SPEC: An ISCA Panel,” *IEEE Micro*, vol. 23, no. 4, 73–77, Jul. 2003, ISSN: 0272-1732. DOI: [10.1109/MM.2003.1225977](https://doi.org/10.1109/MM.2003.1225977).
 - [79] A. Phansalkar, A. Joshi, and L. K. John, “Analysis of Redundancy and Application Balance in the SPEC CPU2006 Benchmark Suite,” in *Proceedings of the 34th Annual International Symposium on Computer Architecture*, ser. ISCA ’07, San Diego, California, USA: ACM, 2007, 412–423, ISBN: 9781595937063. DOI: [10.1145/1250662.1250713](https://doi.org/10.1145/1250662.1250713).
 - [80] R. Giladi and N. Ahitav, “SPEC as a Performance Evaluation Measure,” *Computer*, vol. 28, no. 8, pp. 33–42, 1995. DOI: [10.1109/2.402073](https://doi.org/10.1109/2.402073).
 - [81] L. Eeckhout, H. Vandierendonck, and K. De Bosschere, “Designing Computer Architecture Research Workloads,” *Computer*, vol. 36, no. 2, pp. 65–71, 2003. DOI: [10.1109/MC.2003.1178050](https://doi.org/10.1109/MC.2003.1178050).
 - [82] Community, *LLM inference in C/C++*, ggml-org, 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp>.
 - [83] Community, *Port of OpenAI’s Whisper model in C/C++*, ggml-org, 2026. [Online]. Available: <https://github.com/ggml-org/whisper.cpp>.
 - [84] Community, *AV1 Codec Library*, Alliance for Open Media, 2026. [Online]. Available: <https://aomedia.googlesource.com/aom/>.
 - [85] Community, *Opus Interactive Audio Codec*, Xiph, 2026. [Online]. Available: www.opus-codec.org.
 - [86] Community, *Brotli compression*, Google, 2026. [Online]. Available: <https://github.com/google/brotli>.
 - [87] Community, *7-zip file archiver with compression*, 2026. [Online]. Available: www.7-zip.org.
 - [88] J. Hall, *HiGHS linear optimization software*, 2026. [Online]. Available: <https://highs.dev/>.
 - [89] Community, *HiGHS as a SPEC CPU benchmark*, ERGO-Code/HiGHS, 2024. [Online]. Available: <https://github.com/ERGO-Code/HiGHS/discussions/1929>.
 - [90] NASA, *NAS Parallel Benchmarks*, 1994. [Online]. Available: www.nas.nasa.gov/software/npb.html.
 - [91] SPEC, *SPECvirt® Datacenter 2021*. [Online]. Available: www.spec.org/virt_datacenter2021.
 - [92] B. Coconnier, *Fix illegal funcptr conversions*, JSBSim, 2025. [Online]. Available: <https://github.com/JSBSim-Team/jsbsim/pull/1304>.
 - [93] M. Madhav, *Fixes for sanitizier errors*, cppcheck, 2025. [Online]. Available: <https://github.com/danmar/cppcheck/pull/7697>.
 - [94] M. Madhav, *Fix for uninitialized variable*, DIAMOND, 2024. [Online]. Available: <https://github.com/bbuchfink/diamond/pull/828>.

- [95] K. Kashyap, *Fix GCC warning and UBSan error*, ntest, 2025. [Online]. Available: <https://github.com/vladpetric/ntest/pull/8>.
- [96] H. Plessner, *Fix issues uncovered by sanitizer*, NEST, 2025. [Online]. Available: <https://github.com/nest/nest-simulator/pull/3460>.
- [97] M. Madhav, *Fix for sanitizer's initialization-order-fiasco*, DIAMOND, 2025. [Online]. Available: <https://github.com/bbuchfink/diamond/pull/877>.
- [98] G. Tack, *Use uint's for vector sizes and capacities*, chuffed, 2024. [Online]. Available: <https://github.com/chuffed/chuffed/pull/203>.
- [99] M. Madhav, *match types in hxt*, Gmsh, 2025. [Online]. Available: https://gitlab.onelab.info/gmsh/gmsh/-/merge_requests/558.
- [100] M. Madhav, *Fix many warnings, courtesy of SPEC CPU development*, Gmsh, 2025. [Online]. Available: https://gitlab.onelab.info/gmsh/gmsh/-/merge_requests/551.
- [101] A. Singer, *Removed Template ID From Constructors*, vtr-verilog-to-routing, 2024. [Online]. Available: <https://github.com/verilog-to-routing/vtr-verilog-to-routing/pull/2824>.
- [102] M. Madhav, *Conform to C++17*, Gmsh, 2022. [Online]. Available: https://gitlab.onelab.info/gmsh/gmsh/-/merge_requests/486.
- [103] P. Barnes, *Replace ns3::NaN with std::nan*, NS3, 2024. [Online]. Available: <https://gitlab.com/nsnam/ns-3-dev/-/issues/1161>.
- [104] S. McLeod, *Consistency for some properties*, JSBSim, 2024. [Online]. Available: <https://github.com/JSBSim-Team/jsbsim/pull/1166>.
- [105] G. Tack, *Big endian fixes*, chuffed, 2024. [Online]. Available: <https://github.com/chuffed/chuffed/pull/201>.
- [106] A. Mishchenko, *Fix big-endian problems in mfs2*, abc, 2024. [Online]. Available: <https://github.com/berkeley-abc/abc/commit/733fec3>.
- [107] J. Lowe-Power, *Fix atomic ops on big endian hosts*, gem5, 2025. [Online]. Available: <https://github.com/gem5/gem5/pull/2143>.
- [108] R. Biener, *Big-endian union bug*, GCC Bugzilla, 2019. [Online]. Available: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=88739.
- [109] Community, *mingw-w64*, MinGW. [Online]. Available: www.mingw-w64.org.
- [110] S. McLeod, *Argument of perigee only valid if both inclination and eccentricity are non-zero*, JSBSim, 2023. [Online]. Available: <https://github.com/JSBSim-Team/jsbsim/pull/981>.
- [111] M. Madhav, *Portable debug flag generation*, gem5, 2024. [Online]. Available: <https://github.com/gem5/gem5/pull/1666>.
- [112] H. Plessner, *Correctly detect that substring was not found*, NEST, 2025. [Online]. Available: <https://github.com/nest/nest-simulator/pull/3423>.
- [113] S. Chittireddy, *Fix size calculation in vbptr split memory region in EmitNullBaseClassInitialization*, llvm-project, 2026. [Online]. Available: <https://github.com/llvm/llvm-project/pull/184558>.
- [114] Strace Project, *Strace: Linux syscall tracer*, GitHub repository, 2025. [Online]. Available: <https://github.com/strace/strace>.
- [115] S. Godard, *Sysstat: The sar utility for Linux performance monitoring*, PERFORMANCESTACK, 2025. [Online]. Available: <https://performancestack.in/sar-in-linux/>.
- [116] Intel Corporation, *EMON User Guide*, Intel Documentation, 2025. [Online]. Available: <https://cdrdv2-public.intel.com/686077/emon-users-guide.pdf>.
- [117] M. Madhav, *Efficient i/o for board printout*, ntest, 2024. [Online]. Available: <https://github.com/vladpetric/ntest/pull/6>.
- [118] A. Singer, *Increased Read Buffers to Reduce Total Syscalls*, vtr-verilog-to-routing, 2024. [Online]. Available: <https://github.com/verilog-to-routing/vtr-verilog-to-routing/pull/2814>.
- [119] M. Madhav, *reduce write syscalls in stats output*, gem5, 2025. [Online]. Available: <https://github.com/gem5/gem5/pull/2578>.
- [120] G. Ferreira, *Keep trace files open*, NS3, 2022. [Online]. Available: https://gitlab.com/nsnam/ns-3-dev/-/merge_requests/814.
- [121] R. N. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzeinghi, S. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk, "It Is Time to Standardize Principles and Practices for Software Memory Safety," *Commun. ACM*, vol. 68, no. 2, 40–45, Jan. 2025, ISSN: 0001-0782. DOI: [10.1145/3708553](https://doi.org/10.1145/3708553).
- [122] A. Lilley Brinker, "Memory Safety for Skeptics," *ACM Queue*, vol. 23, no. 5, Nov. 2025, ISSN: 1542-7730. [Online]. Available: <https://spawn-queue.acm.org/doi/10.1145/3773095>.
- [123] M. Shavrick, A. Rebert, L. Dionne, and K. Varlamov, "Practical Security in Production," *ACM Queue*, vol. 23, no. 5, Nov. 2025, ISSN: 1542-7730. [Online]. Available: <https://spawn-queue.acm.org/doi/10.1145/3773097>.
- [124] B. Lord, "The Urgent Need for Memory Safety in Software Products," *Cybersecurity & Infrastructure Security Agency*, 2023. [Online]. Available: www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products.
- [125] The White House, *Back to the building blocks: A path toward secure and measurable software*, 2024. [Online]. Available: <https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf>.
- [126] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A Fast Address Sanity Checker," in *Proceedings of the 2012 USENIX Conference on Annual Technical Conference*, ser. USENIX ATC'12, Boston, MA: USENIX Association, 2012, p. 28. [Online]. Available: <https://dl.acm.org/doi/10.5555/2342821.2342849>.
- [127] G. Saileshwar, R. Boivie, T. Chen, B. Segal, and A. Buyuktosunoglu, "HeapCheck: Low-cost Hardware Support for Memory Safety," *ACM Trans. Archit. Code Optim.*, vol. 19, no. 1, Jan. 2022, ISSN: 1544-3566. DOI: [10.1145/3495152](https://doi.org/10.1145/3495152).
- [128] F. Gorter, E. Barberis, R. Isemann, E. Van Der Kouwe, C. Giuffrida, and H. Bos, "FloatZone: Accelerating Memory Error Detection Using the Floating Point Unit," in *Proceedings of the 32nd USENIX Conference on Security Symposium*, ser. SEC '23, Anaheim, CA, USA: USENIX Association, 2023, ISBN: 978-1-939133-37-3. [Online]. Available: www.usenix.org/conference/usenixsecurity23/presentation/gorter.
- [129] "Program Instrumentation Options." [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html>.
- [130] "Clang 22.0.0 git documentation: Address Sanitizer." [Online]. Available: <https://clang.llvm.org/docs/AddressSanitizer.html>.
- [131] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data Race Detection in Practice," in *Proceedings of the Workshop on Binary Instrumentation and Applications*, ser. WBIA '09, New York, New York, USA: ACM, 2009, 62–71, ISBN: 9781605587936. DOI: [10.1145/1791194.1791203](https://doi.org/10.1145/1791194.1791203).
- [132] S. Kaushik, M. Madhav, N. Aboulenein, J. Bessette, S. Brahmadathan, B. Chaffin, M. Erler, S. Jourdan, T. Maciukenas, R. Masti, J. Perry, M. Suter, S. Tetric, B. Toll, D. Turley, C. Worth, and A. Bajwa, "Optimized Memory Tagging on AmpereOne Processors," in *Proceedings of the 53rd Annual International Symposium on Computer Architecture*, ser. ISCA 2026, Raleigh, NC, USA: IEEE, 2026. arXiv: [2511.17773](https://arxiv.org/abs/2511.17773) [cs.AR].
- [133] M. Madhav, *Fix memory safety issue*, NEST, 2024. [Online]. Available: <https://github.com/nest/nest-simulator/pull/3272>.
- [134] M. Madhav, *Sanitizer fixes from SPEC CPU dev*, gem5, 2025. [Online]. Available: <https://github.com/gem5/gem5/pull/2531>.
- [135] H. Plessner, *Fix data race in NodeManager*, NEST, 2025. [Online]. Available: <https://github.com/nest/nest-simulator/pull/3575>.
- [136] C. Geuzaine, *Fix omp data race in getBasis*, Gmsh, 2025. [Online]. Available: <https://gitlab.onelab.info/gmsh/gmsh/-/commit/cf2bb38a>.
- [137] Ampere Computing, *Lenovo ThinkSystem HR330A*, 2019. [Online]. Available: <https://amperecomputing.com/customer-connect/products/lenovo-hr330a>.
- [138] Ampere Computing, *Ampere® eMAG® 8180 Product Brief*, 2020. [Online]. Available: <https://amperecomputing.com/customer-connect/products/emag-device-documentation>.
- [139] T. M. Conte and W. W. Hwu, "Benchmark characterization," *Proceedings of the Twenty-Fourth Annual Hawaii International Conference on System Sciences*, vol. i, 365–372 vol.1, 1991. [Online]. Available: <https://api.semanticscholar.org/CorpusID:195840684>.
- [140] J. L. Henning, "Performance Counters and Development of SPEC CPU2006," *SIGARCH Comput. Archit. News*, vol. 35, no. 1, Mar. 2007, ISSN: 0163-5964. DOI: [10.1145/1241601.1241623](https://doi.org/10.1145/1241601.1241623).
- [141] A. Kejariwal, A. V. Veidenbaum, A. Nicolau, X. Tian, M. Girkar, H. Saito, and U. Banerjee, "Comparative Architectural Characterization of SPEC CPU2000 and CPU2006 Benchmarks on the Intel Core 2 Duo Processor," in *2008 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation*, 2008, pp. 132–141. DOI: [10.1109/ICSAMOS.2008.4664856](https://doi.org/10.1109/ICSAMOS.2008.4664856).
- [142] A. Limaye and T. Adegbeja, "A Workload Characterization of the SPEC CPU2017 Benchmark Suite," in *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2018, pp. 149–158. DOI: [10.1109/ISPASS.2018.00028](https://doi.org/10.1109/ISPASS.2018.00028).

- [143] M. Hassan, C. H. Park, and D. Black-Schaffer, "A Reusable Characterization of the Memory System Behavior of SPEC2017 and SPEC2006," *ACM Trans. Archit. Code Optim.*, vol. 18, no. 2, Mar. 2021, ISSN: 1544-3566. DOI: [10.1145/3446200](https://doi.org/10.1145/3446200).
- [144] A. Phansalkar, A. Joshi, and L. K. John, "Subsetting the SPEC CPU2006 Benchmark Suite," *SIGARCH Comput. Archit. News*, vol. 35, no. 1, 69–76, Mar. 2007, ISSN: 0163-5964. DOI: [10.1145/1241601.1241616](https://doi.org/10.1145/1241601.1241616).
- [145] S. Singh and M. Awasthi, "Memory Centric Characterization and Analysis of SPEC CPU2017 Suite," in *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '19, Mumbai, India: ACM, 2019, ISBN: 9781450362399. DOI: [10.1145/3297663.3310311](https://doi.org/10.1145/3297663.3310311).
- [146] A. Navarro-Torres, J. Alastruey-Benedé, P. Ibáñez-Marín, and V. Viñals-Yúfera, "Memory Hierarchy Characterization of SPEC CPU2006 and SPEC CPU2017 on the Intel Xeon Skylake-SP," *PLOS ONE*, vol. 14, no. 8, pp. 1–24, Aug. 2019. DOI: [10.1371/journal.pone.0220135](https://doi.org/10.1371/journal.pone.0220135).
- [147] A. Yasin, "A Top-Down Method for Performance Analysis and Counters Architecture," in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2014, pp. 35–44. DOI: [10.1109/ISPASS.2014.6844459](https://doi.org/10.1109/ISPASS.2014.6844459).
- [148] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, "Automatically Characterizing Large Scale Program Behavior," in *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS X, San Jose, California: ACM, 2002, 45–57, ISBN: 1581135742. DOI: [10.1145/605397.605403](https://doi.org/10.1145/605397.605403).
- [149] V. Weaver, *BBV: basic block vector generation*, Valgrind. [Online]. Available: www.valgrind.org/docs/manual/bbv-manual.html.
- [150] S. Caculo, M. Madhav, and J. Baxter, "Memory Access Vectors: Improving Sampling Fidelity for CPU Performance Simulations," in *ARM-based General-Purpose Computing (ISCA'25 Workshop)*, Tokyo, Japan, 2025. arXiv: [2506.02344](https://arxiv.org/abs/2506.02344) [cs.AR].
- [151] J. L. Gustafson, "Reevaluating Amdahl's Law," *Commun. ACM*, vol. 31, no. 5, 532–533, May 1988, ISSN: 0001-0782. DOI: [10.1145/42411.42415](https://doi.org/10.1145/42411.42415).
- [152] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, ser. AFIPS '67 (Spring), Atlantic City, New Jersey: ACM, 1967, 483–485, ISBN: 9781450378956. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560).
- [153] A. Carlton, *CINT92 and CFP 92 Homogeneous Capacity Method Offers Fair Measure of Processing Capacity*, 1995. [Online]. Available: www.spec.org/cpu92/specrate.txt.
- [154] R. A. Velásquez, P. Michaud, and A. Sez nec, "Selecting Benchmark Combinations for the Evaluation of Multicore Throughput," in *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2013, pp. 173–182. DOI: [10.1109/ISPASS.2013.6557168](https://doi.org/10.1109/ISPASS.2013.6557168).
- [155] S. Imtiaz, M. Behnam, G. Capannini, J. Carlson, and M. Jägemar, "Predicting Execution Time of Concurrent Applications Using Performance Counters," in *2025 IEEE International Conference on Industrial Technology (ICIT)*, 2025, pp. 1–7. DOI: [10.1109/ICIT63637.2025.10965122](https://doi.org/10.1109/ICIT63637.2025.10965122).
- [156] S. K. Shukla, M. Shukla, U. Dwivedi, and B. Pant, "An Investigation and Analysis of Interference in Multicore Environment," in *2022 IEEE 11th International Conference on Communication Systems and Network Technologies (CSNT)*, 2022, pp. 154–159. DOI: [10.1109/CSNT54456.2022.9787630](https://doi.org/10.1109/CSNT54456.2022.9787630).
- [157] M. Navarro, J. Feliu, S. Petit, M. E. Gómez, and J. Sahuquillo, "SYNPA: SMT Performance Analysis and Allocation of Threads to Cores in ARM Processors," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2024, pp. 705–715. DOI: [10.1109/IPDPS57955.2024.00068](https://doi.org/10.1109/IPDPS57955.2024.00068).
- [158] P. Petrica, A. M. Izraelevitz, D. H. Albonesi, and C. A. Shoemaker, "Flicker: A Dynamically Adaptive Architecture for Power Limited Multicore Systems," in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, ser. ISCA '13, Tel-Aviv, Israel: ACM, 2013, 13–23, ISBN: 9781450320795. DOI: [10.1145/2485922.2485924](https://doi.org/10.1145/2485922.2485924).
- [159] L. Pons, J. Sahuquillo, V. Selfa, S. Petit, and J. Pons, "Phase-Aware Cache Partitioning to Target Both Turnaround Time and System Performance," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 11, 2020. DOI: [10.1109/TPDS.2020.2996031](https://doi.org/10.1109/TPDS.2020.2996031).
- [160] A. N. Jacobvitz, A. D. Hilton, and D. J. Sorin, "Multi-program Benchmark Definition," in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 72–82. DOI: [10.1109/ISPASS.2015.7095786](https://doi.org/10.1109/ISPASS.2015.7095786).
- [161] M. Van Biesbrouck, L. Eeckhout, and B. Calder, "Representative Multiprogram Workloads for Multithreaded Processor Simulation," in *2007 IEEE 10th International Symposium on Workload Characterization*, 2007, pp. 193–203. DOI: [10.1109/IISWC.2007.4362195](https://doi.org/10.1109/IISWC.2007.4362195).
- [162] S. Eyerman and L. Eeckhout, "System-Level Performance Metrics for Multiprogram Workloads," *IEEE Micro*, vol. 28, no. 3, pp. 42–53, 2008. DOI: [10.1109/MM.2008.44](https://doi.org/10.1109/MM.2008.44).
- [163] S. Eyerman, P. Michaud, and W. Rogiest, "Multiprogram Throughput Metrics: A Systematic Approach," *ACM Transactions on Architecture and Code Optimization*, vol. 11, no. 3, Oct. 2014, ISSN: 1544-3566. DOI: [10.1145/2663346](https://doi.org/10.1145/2663346).
- [164] V. Selfa, J. Sahuquillo, C. Gómez, and M. E. Gómez, "Methodologies and Performance Metrics to Evaluate Multiprogram Workloads," in *Proceedings of the 2015 23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, ser. PDP '15, USA: IEEE Computer Society, 2015, 150–154, ISBN: 9781479984916. DOI: [10.1109/PDP.2015.74](https://doi.org/10.1109/PDP.2015.74).
- [165] E. Tomusk, C. Dubach, and M. O'boyle, "Four Metrics to Evaluate Heterogeneous Multicores," *ACM Trans. Archit. Code Optim.*, vol. 12, no. 4, Nov. 2015, ISSN: 1544-3566. DOI: [10.1145/2829950](https://doi.org/10.1145/2829950).
- [166] A. Hilton, N. Eswaran, and A. Roth, "FIESTA: A Sample-Balanced Multi-Program Workload Methodology," *Workshop on Modeling, Benchmarking and Simulation (MoBS)*, Jul. 2009. [Online]. Available: www.researchgate.net/publication/228579796_FIESTA_A_Sample-Balanced_Multi-Program_Workload_Methodology.
- [167] SPEC, *docs: Rolling Round-Robin rate*, 2026. [Online]. Available: <https://www.spec.org/cpu2026/Docs/runcpu.html#section1.8>.
- [168] T. Yu, R. Zhong, V. Janjic, P. Petoumenos, J. Zhai, H. Leather, and J. Thomson, "Collaborative Heterogeneity-Aware OS Scheduler for Asymmetric Multicore Processors," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 5, pp. 1224–1237, 2021. DOI: [10.1109/TPDS.2020.3045279](https://doi.org/10.1109/TPDS.2020.3045279).
- [169] T. Creech, A. Kotha, and R. Barua, "Efficient Multiprogramming for Multicores with SCAF," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-46, Davis, California: ACM, 2013, 334–345, ISBN: 9781450326384. DOI: [10.1145/2540708.2540737](https://doi.org/10.1145/2540708.2540737).
- [170] R. B. Roy, T. Patel, and D. Tiwari, "SATORI: Efficient and Fair Resource Partitioning by Sacrificing Short-Term Benefits for Long-Term Gains," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 292–305. DOI: [10.1109/ISCA52012.2021.00031](https://doi.org/10.1109/ISCA52012.2021.00031).
- [171] P. Zou, X. Feng, and R. Ge, "Contention Aware Workload and Resource Co-Scheduling on Power-Bounded Systems," in *2019 IEEE International Conference on Networking, Architecture and Storage (NAS)*, 2019, pp. 1–8. DOI: [10.1109/NAS.2019.8834721](https://doi.org/10.1109/NAS.2019.8834721).
- [172] P. Prieto, P. Abad, J. A. Gregorio, and V. Puente, "Fast, Accurate Processor Evaluation Through Heterogeneous, Sample-Based Benchmarking," *IEEE Transactions on Parallel & Distributed Systems*, vol. 32, no. 12, pp. 2983–2995, Dec. 2021, ISSN: 1558-2183. DOI: [10.1109/TPDS.2021.3080702](https://doi.org/10.1109/TPDS.2021.3080702).
- [173] P. Prieto, P. Abad, J. A. Herrero, J. A. Gregorio, and V. Puente, "SPECcast: A Methodology for Fast Performance Evaluation with SPEC CPU 2017 Multiprogrammed Workloads," in *Proceedings of the 49th International Conference on Parallel Processing*, ser. ICPP '20, Edmonton, AB, Canada: ACM, 2020, ISBN: 9781450388160. DOI: [10.1145/3404397.3404424](https://doi.org/10.1145/3404397.3404424).
- [174] I. Sutherland, "Technology and Courage," in *CMU Computer Science, A 25th Anniversary Commemorative*, New York, NY, USA: Association for Computing Machinery, 1991, pp. 425–447, ISBN: 0201528991. DOI: [10.1145/107219](https://doi.org/10.1145/107219). [Online]. Available: <https://labs.oracle.com/pls/apex/f?p=94065:10:112314537568218:2628>.